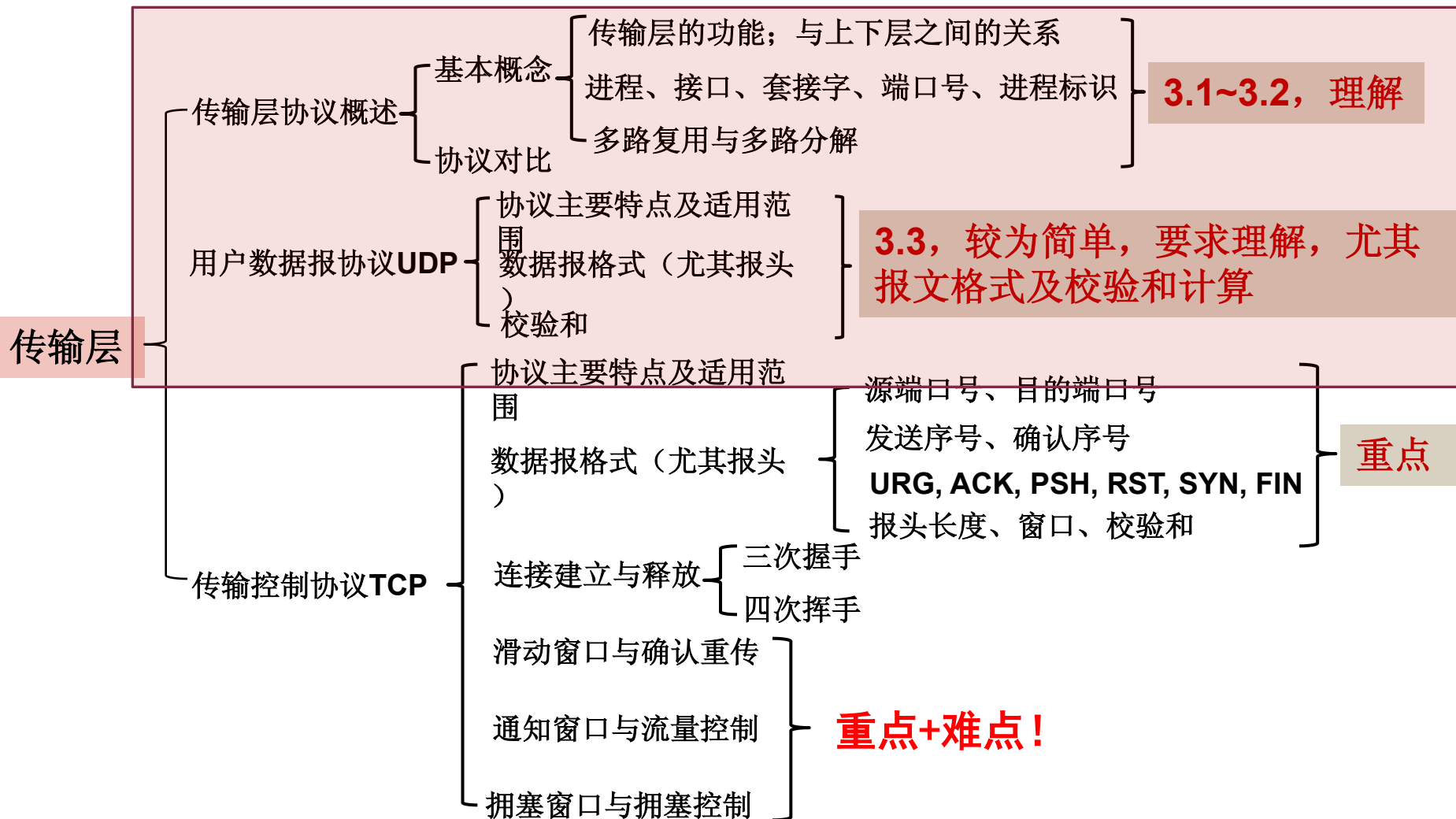


前讲复习



前讲复习

1. UDP报文头标不包括（ ）。
A. 目的地址 B. 源UDP端口 C. 目的UDP端口 D. 报文长度
2. 下列关于TCP和UDP的描述正确的是（ ）。
A. TCP和UDP都是无连接的
B. TCP是无连接的，UDP面向连接
C. TCP适用于可靠性较差的广域网，UDP适用于可靠性较高的局域网
D. TCP适用于可靠性较高的局域网，UDP适用于可靠性较差的广域网
3. UDP 具有以下（ ）特征。
A. 在服务器上维护连接状态信息 B. 通过三次握手建立连接
C. 调节发送速率 D. 以上都不是
4. 如果用户程序使用UDP协议进行数据传输，那么（ ）协议必须承担可靠性方面的全部工作。
A. 数据链路层 B. 网络层 C. 传输层 D. 应用层



前讲复习

1. UDP报文头标不包括（ A ）。
A. 目的地址 B. 源UDP端口 C. 目的UDP端口 D. 报文长度
2. 下列关于TCP和UDP的描述正确的是（ C ）。
A. TCP和UDP都是无连接的
B. TCP是无连接的，UDP面向连接
C. TCP适用于可靠性较差的广域网，UDP适用于可靠性较高的局域网
D. TCP适用于可靠性较高的局域网，UDP适用于可靠性较差的广域网
3. UDP 具有以下（ D ）特征。
A. 在服务器上维护连接状态信息 B. 通过三次握手建立连接
C. 调节发送速率 D. 以上都不是
4. 如果用户程序使用UDP协议进行数据传输，那么（ D ）协议必须承担可靠性方面的全部工作。
A. 数据链路层 B. 网络层 C. 传输层 D. 应用层

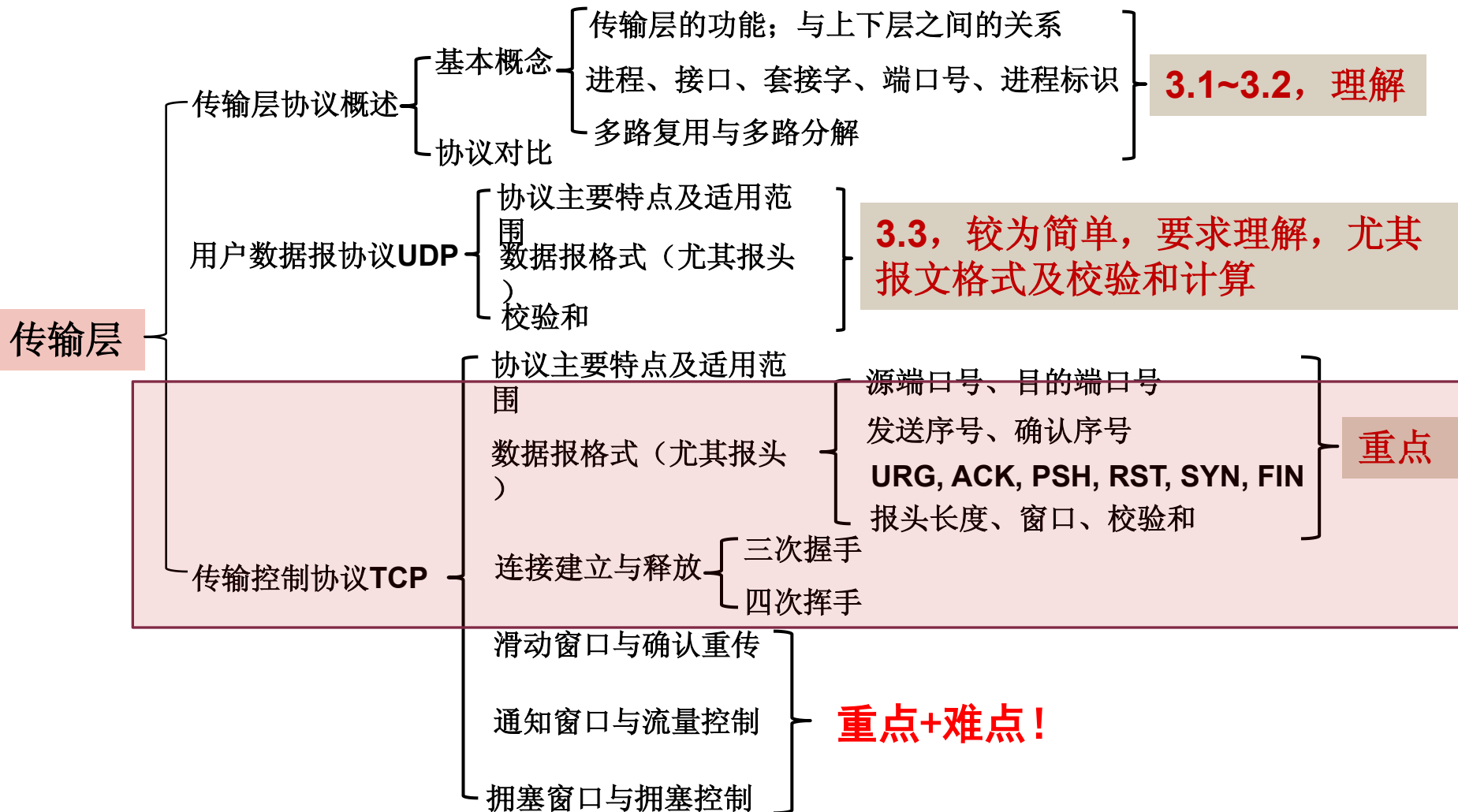


传输层-TCP

计算机网络



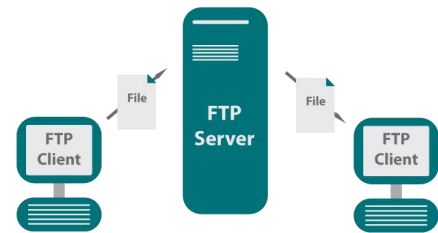
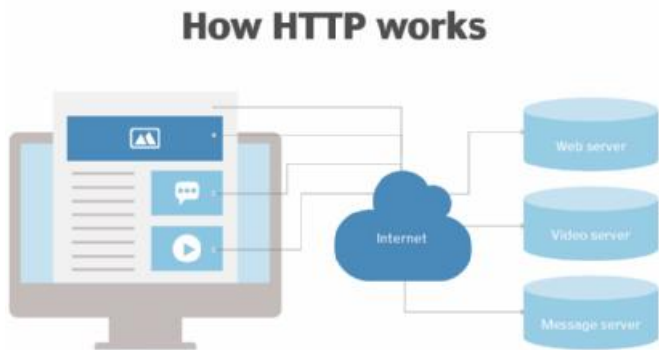
本讲概述



3.4 传输控制协议

3.4.1 TCP协议的主要特点及应用

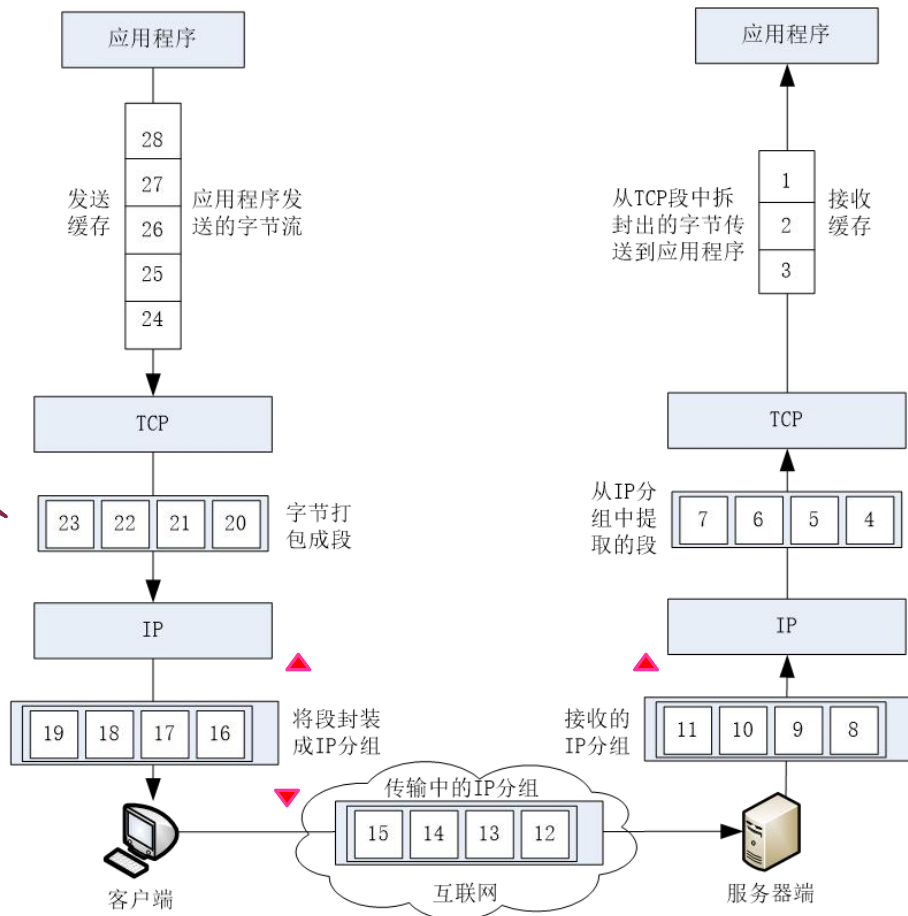
- 支持**面向连接**的服务：打电话式、会话式通信
- 支持**字节流**传输：字节管道、字节按序传输和到达
- 支持**全双工**服务：一个应用进程可以同时收发数据、**捎带确认**
- 支持建立多个并发的TCP连接（服务器同时响应多个连接）
- 支持可靠传输服务：不丢失、不重复、有序



3.4 传输控制协议

3.4.1 TCP协议的主要特点

注意：TCP传输单元是“报文段”（Segment）

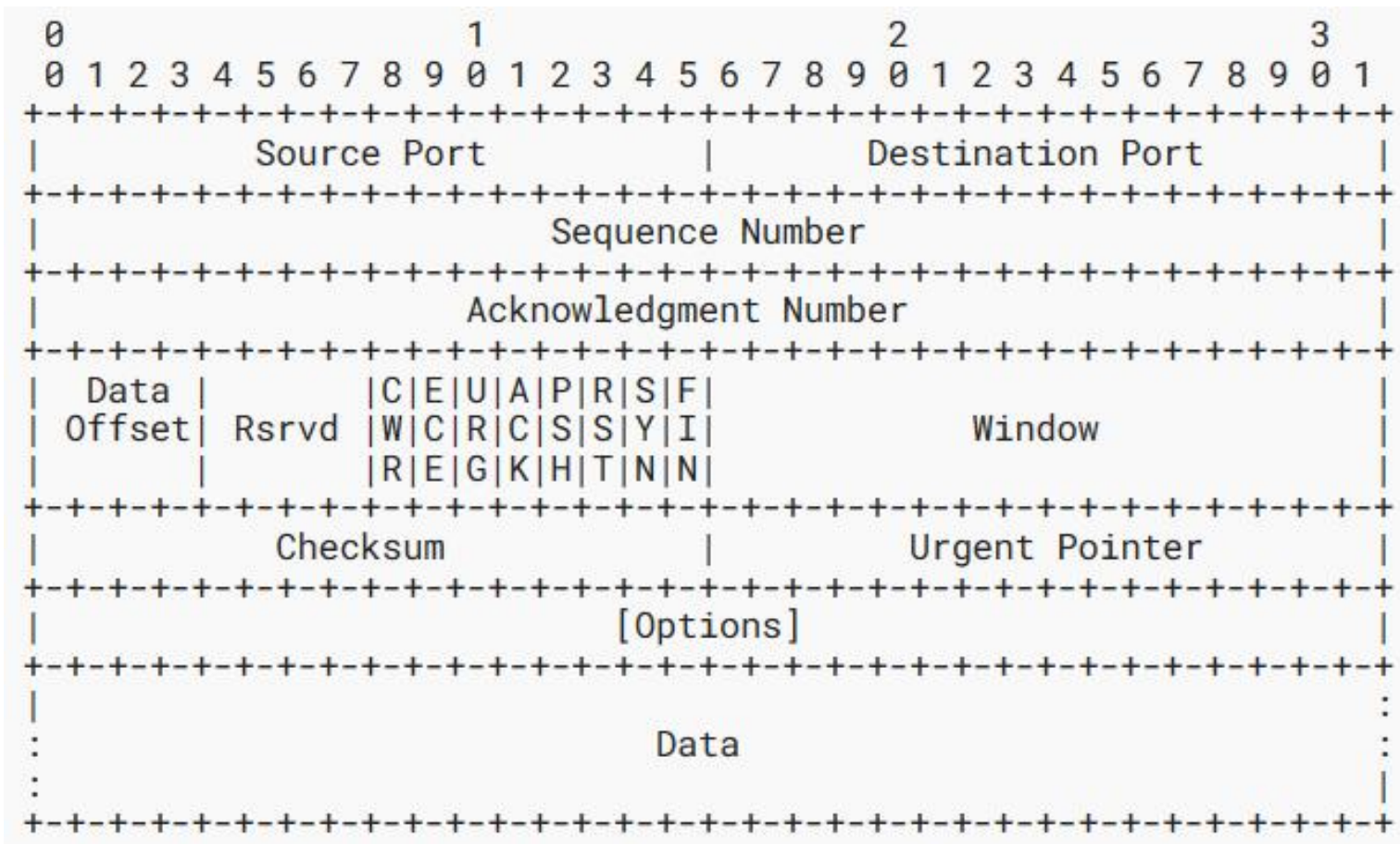


需要的机制

- 建立连接
- 可靠数据传输
- 流量控制
- 拥塞控制

3.4 传输控制协议

3.4.2 TCP报文格式



3.4 传输控制协议

3.4.2 TCP报文格式



- **发送序号:** 若SYN=1,该字段表示TCP数据字段的第1个字节A在当前发送信道T中的初始序号; 若SYN=0, 表示A在T中的序号(大于初始序号)
 - 连接建立时, 初始序号 (ISN) 由**随机**数生成器生成, 发送端和接收端**独立产生**, 可能不一样
 - 占32bit, 所能表示的序号范围是: $0 \sim (2^{32}-1)$

3.4 传输控制协议

3.4.2 TCP报文格式



- **确认序号:** 只有当ACK位=1时有效，表示发送此报文段的进程期望接收的下一个新字节的序号。
 - 确认序号=N+1，表示接收方已经成功接收了序号为N及之前的所有字节，要求发送方接下来应该发送起始序号为N+1的字节段。

3.4 传输控制协议

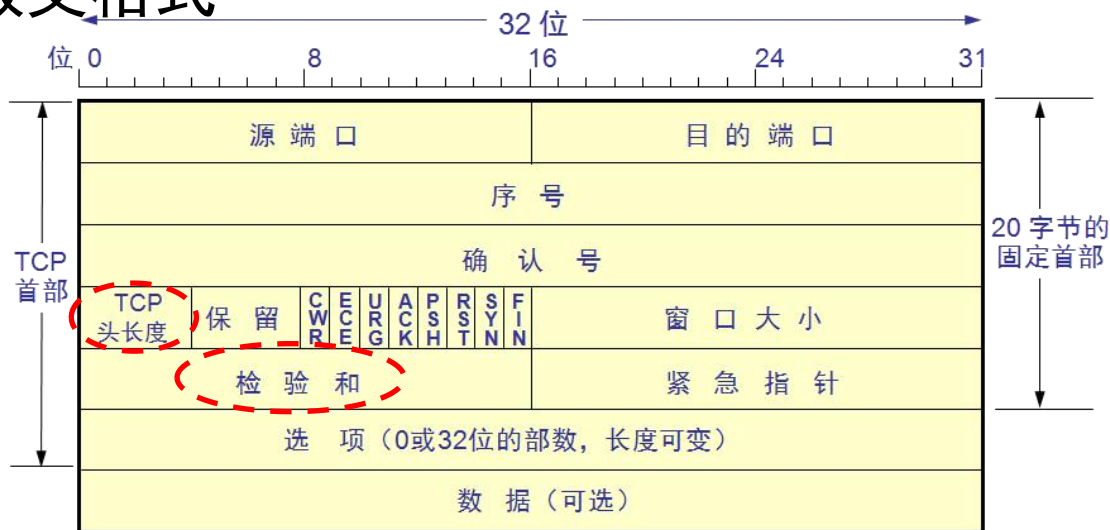
3.4.2 TCP报文格式



- 窗口大小：指示发送当前报文段的进程可以接收的数据长度 (单位: 字节)。
 - 准备接收下一个TCP报文的接收方，通知即将发送报文的发送方下一个报文中最多可以发送的字节数，是发送方确定发送窗口的依据，是动态可变的。
 - 举例：若节点A发送给节点B的TCP报文报头中，确认号字段值为502，窗口字段值为1000，则表示：下一次B向A发送的TCP报文数据字段第一个字节的编号为502，最后一个字节的编号最大为1501

3.4 传输控制协议

3.4.2 TCP报文格式



- 报头长度：单位是4字节，表示TCP报文首部的大小；取值范围[5,15]
(Why?)
- 校验和：
 - 与UDP校验和的相同点：1) 计算方式相同；2) 也需要伪首部。
 - 不同点：1) UDP校验和可选，TCP校验和必须；2) 伪首部协议字段值为6

3.4 传输控制协议

3.4.2 TCP报文格式

标志

说 明

SYN

当 $\text{SYN}=1$ ，而 $\text{ACK}=0$ 时，表明这是一个建立连接请求报文，若对方同意建立该连接，则应在发回的报文中将 SYN 和 ACK 标志位同时置1。实质上，就是用 SYN 来代表 Connection Request和Connection Accepted，用 ACK 位来区分这两种情况。

ACK

确认号字段的值有效。只有当 $\text{ACK}=1$ 时，确认序号字段才有意义。当 $\text{ACK}=0$ 时，确认序号没有意义。

FIN

终止连接。当 $\text{FIN}=1$ 时，表明数据已经发送完毕，并请求释放连接。

RST

连接必须复位。当 $\text{RST}=1$ 时，表明出现严重差错，必须释放连接，然后重新建立连接。

URG

此报文是紧急数据，应尽快传送出去。此标志位要与紧急指针字段配合使用，由紧急指针指出在本报文段中的紧急数据的最后一个字节的编号。

PSH

将数据推向前。当 $\text{PSH}=1$ 时，请求接收方TCP软件将该报文立即推送给应用程序。



3.4 传输控制协议

3.4.2 TCP报文格式扩展 (*)

标志	说 明
ECE	Explicit Congestion Notification Echo 用于显示拥塞通知，表示包带有拥塞通知信息。当路由器或交换机检测到网络拥塞时，会设置ECE标志位，通知TCP连接的两端网络出现拥塞情况。
CWR	Congestion Window Reduced 当接收方收到设置了ECE标志位的数据包时，会向发送方发送带有CWR标志位的确认包，表示接收方的拥塞窗口已减小。



3.4 传输控制协议

■ 3.4.2 TCP报文格式

- TCP最大段长度（MSS）
- 定义：TCP报文数据部分的最大长度，**不包括TCP报头长度**
- 区别：MSS与窗口长度
- 大小选择：1) 考虑协议开销，不能太小；2) 考虑分段导致的网络层开销和传输出错概率，不能太大；3) 考虑发送和接收缓冲区的限制
- 默认值：536字节

Q：为什么说TCP面向字节流而UDP面向报文？



3.4 传输控制协议

3.4.3 TCP基本通信过程

➤ 建立连接阶段

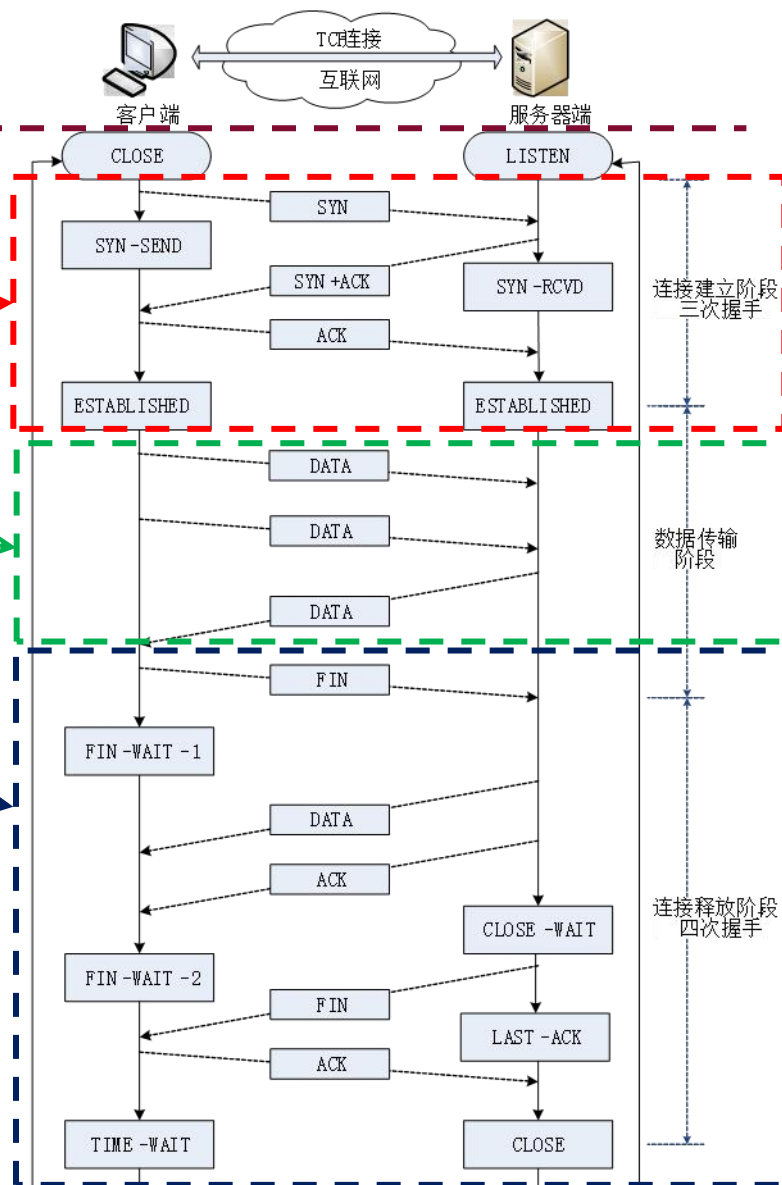
- 3个报文段交互

➤ 数据双向传输阶段

- 以滑动窗口中的多个报文段为单位的双向传输

➤ 释放连接阶段

- 两个方向的连接，可以间隔开来释放
- 每个方向的连接释放，需要2个报文段

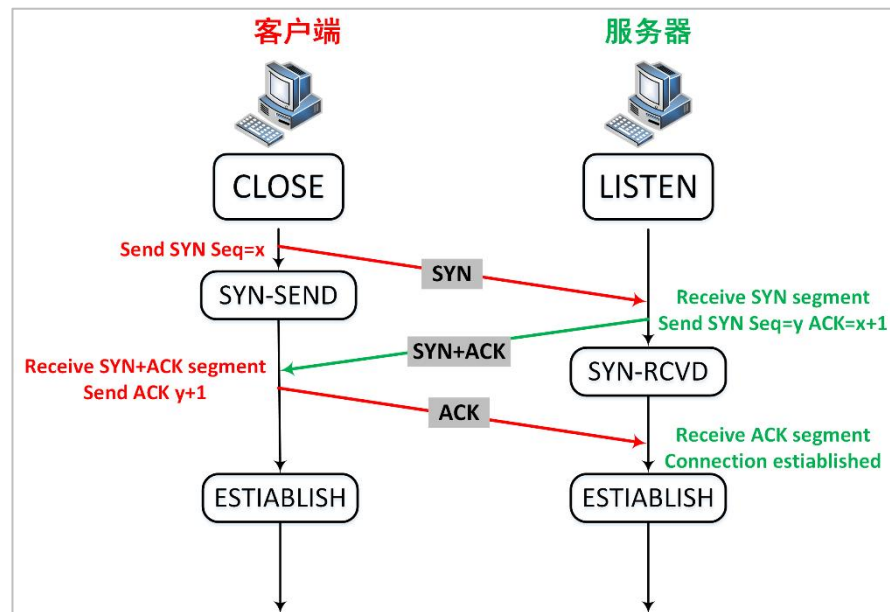


3.4 传输控制协议

3.4.3 TCP基本通信过程

建立连接阶段

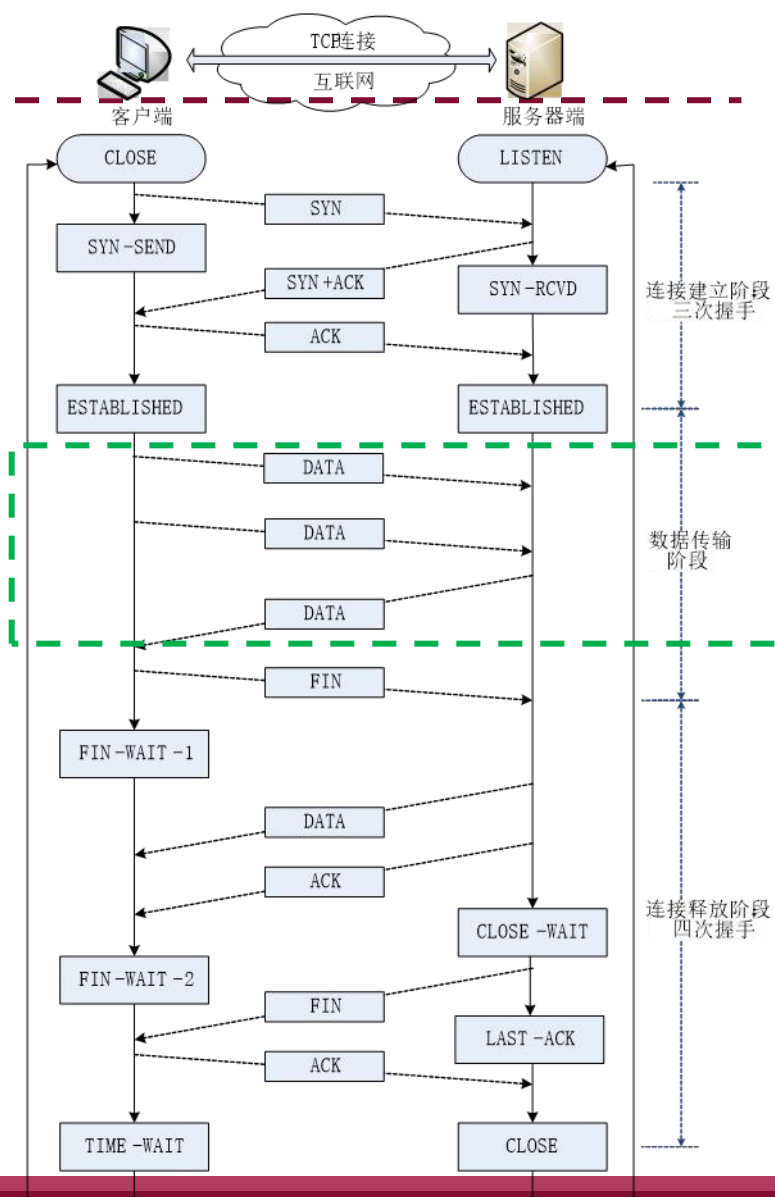
- 客户端：发送连接建立请求， $SYN=1$, $ACK=0$, $Seq=x$ 。SYN=1的报文段**不能携带数据**，但要**消耗掉一个序号**
- 服务器端：收到请求并同意， $SYN=1$, $ACK=1$, $Seq=y$, $ackn=x+1$ 。同样该报文段也是SYN=1的报文段，不能携带数据，但同样要**消耗掉一个序号**。
- 客户端：收到服务器的确认，并对此进行确认。 $ACK=1$, $Seq=x+1$, $ackn=y+1$



思考：为什么需要三次握手建立连接？两次可不可以？为什么？

3.4 传输控制协议

- 3.4.3 TCP基本通信过程
- 数据双向传输阶段
 - 以滑动窗口中的多个报文段为单位的
双向传输
 - 应用进程将数据以字节流发送，无需
考虑发送数据字节长度，由TCP负责将
字节流分段打包
 - 借助滑动窗口，实现按序的，无差错、
不丢失、不重复传送字节流
 - 提供差错控制功能，保证正确接收字
节流（通过差错检测、确认、重传实
现）

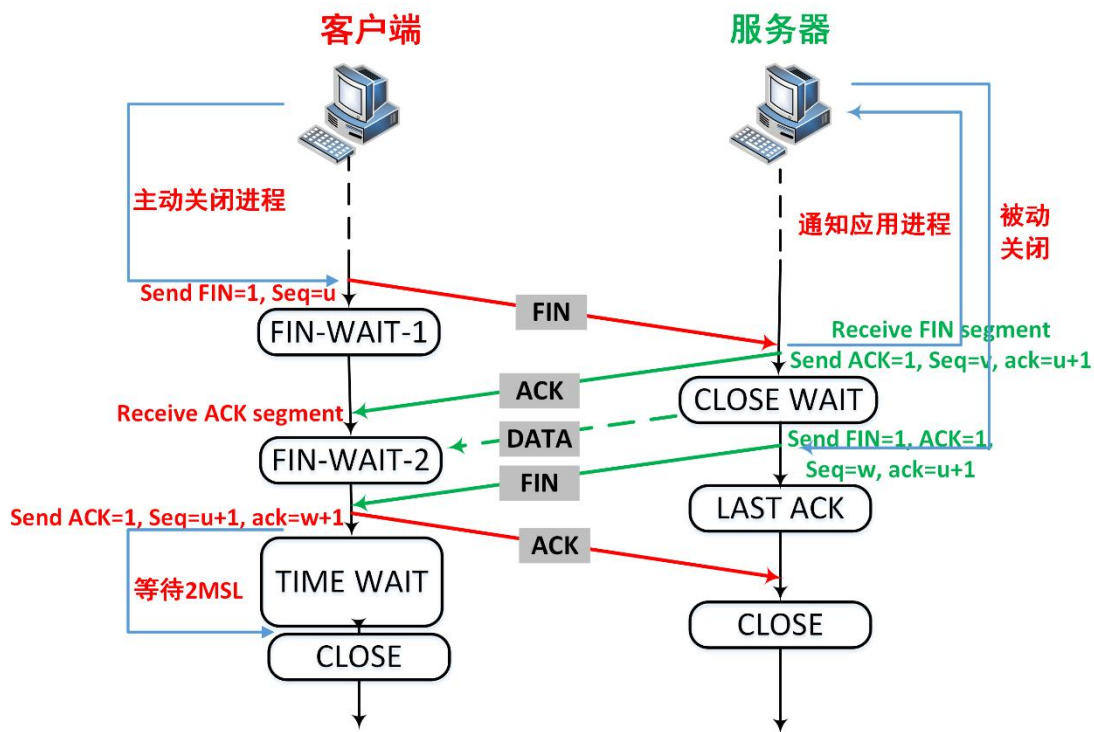


3.4 传输控制协议

3.4.3 TCP基本通信过程

■ 释放连接阶段

- 两个方向的连接，可以间隔开来释放
- 每个方向的连接释放，需要2个报文段
- MSL: Maximum Segment Lifetime, 最大报文生存时间，是任何报文段被丢弃前在网络内的最长时间



思考：为什么需要等待2MSL?

尽管有时FIN报文的数据字段长度=0，但FIN报文仍需要消耗额外1个序列号

3.4 传输控制协议

3.4.3 TCP基本通信过程

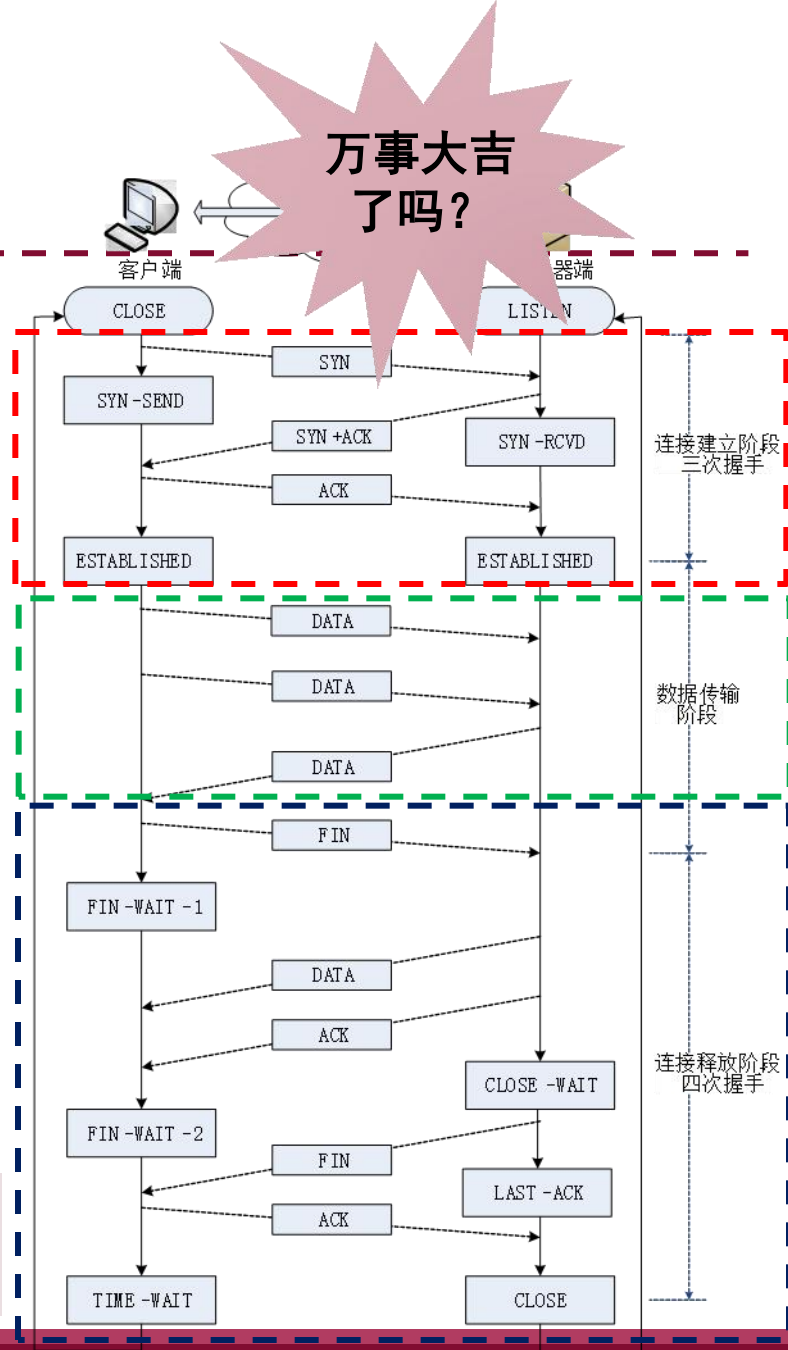
思考以下两个问题：

Q1: 假如客户端建立了到服务器的连接，传输了一些数据，但是突发故障崩溃。此种情况下，服务器将处于一种什么状态？

Q2: 客户端最终进入的TIME-WAIT状态该如何实现？

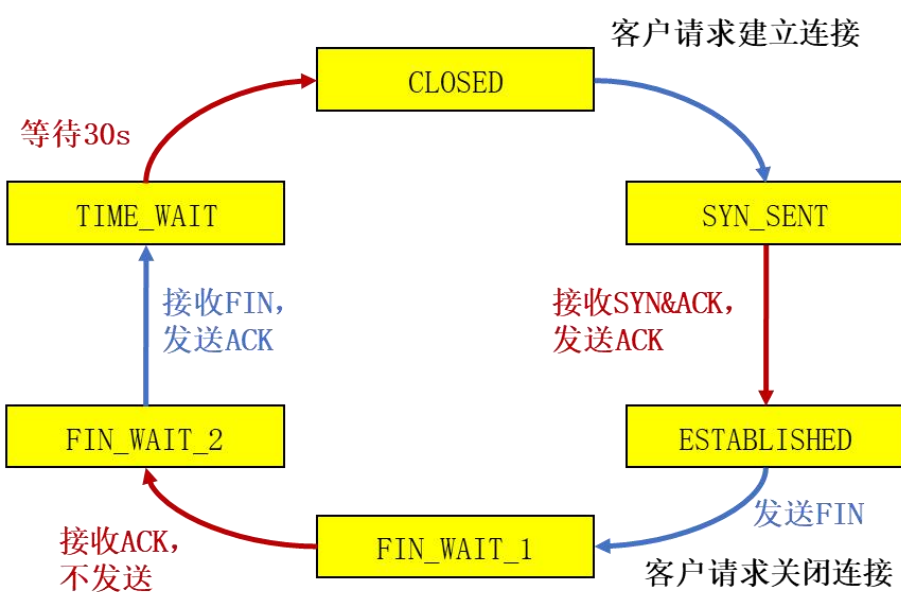
保活定时器（服务器端）：每次收到客户端发来的消息就复位；定时器设定为2小时，超时则发送探测包。连续十个探测包还没收到回应，则中断连接。

时间等待定时器（客户端）：设定为报文寿命的2倍

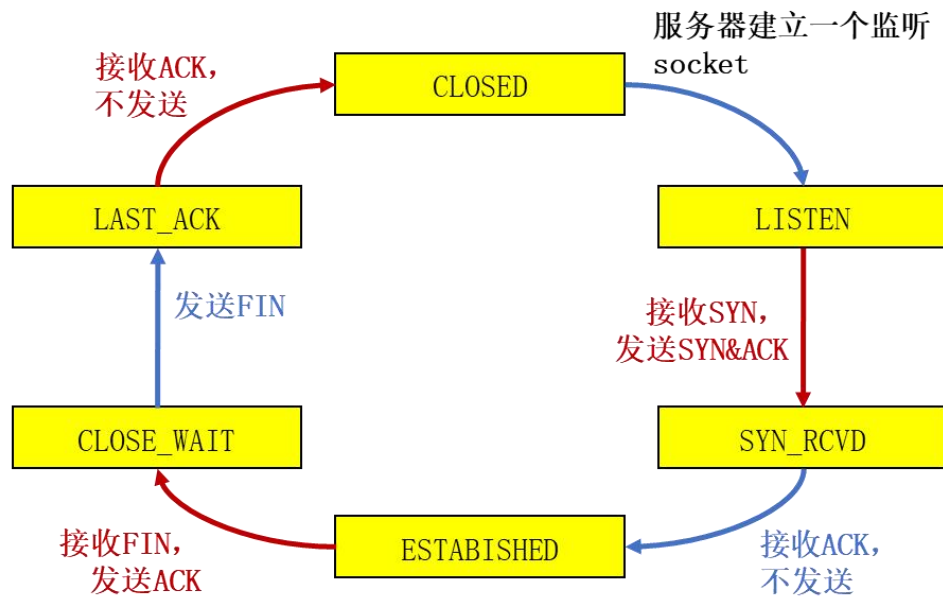


3.4 传输控制协议

3.4.3 TCP基本通信过程



客户生命周期



服务器生命周期

TIME_WAIT与具体实现有关, 一般是30秒、1分钟或2分钟

3.4 传输控制协议

- TCP端口扫描的原理：
 - 扫描程序依次与目标机器的各个端口建立TCP连接
 - 根据获得的响应来收集目标机器信息
- 在典型的TCP端口扫描过程中，发送端向目标端口发送SYN报文段：
 - 若收到SYNACK段，表明目标端口上有服务在运行
 - 若收到RST段，表明目标端口上没有服务在运行
 - 若什么也没收到，表明路径上有防火墙，有些防火墙会丢弃来自外网的SYN报文段
- SYN洪泛攻击：
 - 攻击者采用伪造的源IP地址，向服务器发送大量的SYN段，却不发送ACK段
 - 服务器为维护一个巨大的半连接表耗尽资源，导致无法处理正常客户的连接请求，表现为服务器停止服务



3.4 传输控制协议

随堂练习

1. A和B之间建立了TCP连接，A向B发送了一个报文段，其中序号字段seq=100，确认号字段ACK=101，数据部分包含50B，那么在B对该报文的确认报文段中（ ）
- A. seq=201, ACK=151 B. seq=101, ACK=101
C. seq=151, ACK=101 D. seq=101, ACK=150
2. 在TCP 协议中，断开连接时需要将（ ）字段中的（ ）标志位置1。
- A. 保留, SYN B. 控制, SYN C. 保留, FIN。 D. 控制, FIN
3. 下列关于TCP连接释放过程，叙述不正确的是（ ）
- A. 通过设置FIN为来表示释放连接
B. 当一方释放连接后另一方即不能继续发送数据
C. 只有双方均释放连接后，该连接才被释放
D. 双方都发送FIN包后，会进入TIME_WAIT状态，等待2MSL时间后才完全关闭连接



3.4 传输控制协议

随堂练习

1. A和B之间建立了TCP连接，A向B发送了一个报文段，其中序号字段seq=100，确认号字段ACK=101，数据部分包含50B，那么在B对该报文的确认报文段中（ **D** ）

A. seq=201, ACK=151

B. seq=101, ACK=101

C. seq=151, ACK=101

D. seq=101, ACK=150

2. 在TCP 协议中，断开连接时需要将（ **D** ）字段中的（ ）标志位置1。

A. 保留, SYN B. 控制, SYN C. 保留, FIN。 D. 控制, FIN

3. 下列关于TCP连接释放过程，叙述不正确的是（ **B** ）

A. 通过设置FIN为来表示释放连接

B. 当一方释放连接后另一方即不能继续发送数据

C. 只有双方均释放连接后，该连接才被释放

D. 双方都发送FIN包后，会进入TIME_WAIT状态，等待2MSL时间后才完全关闭连接



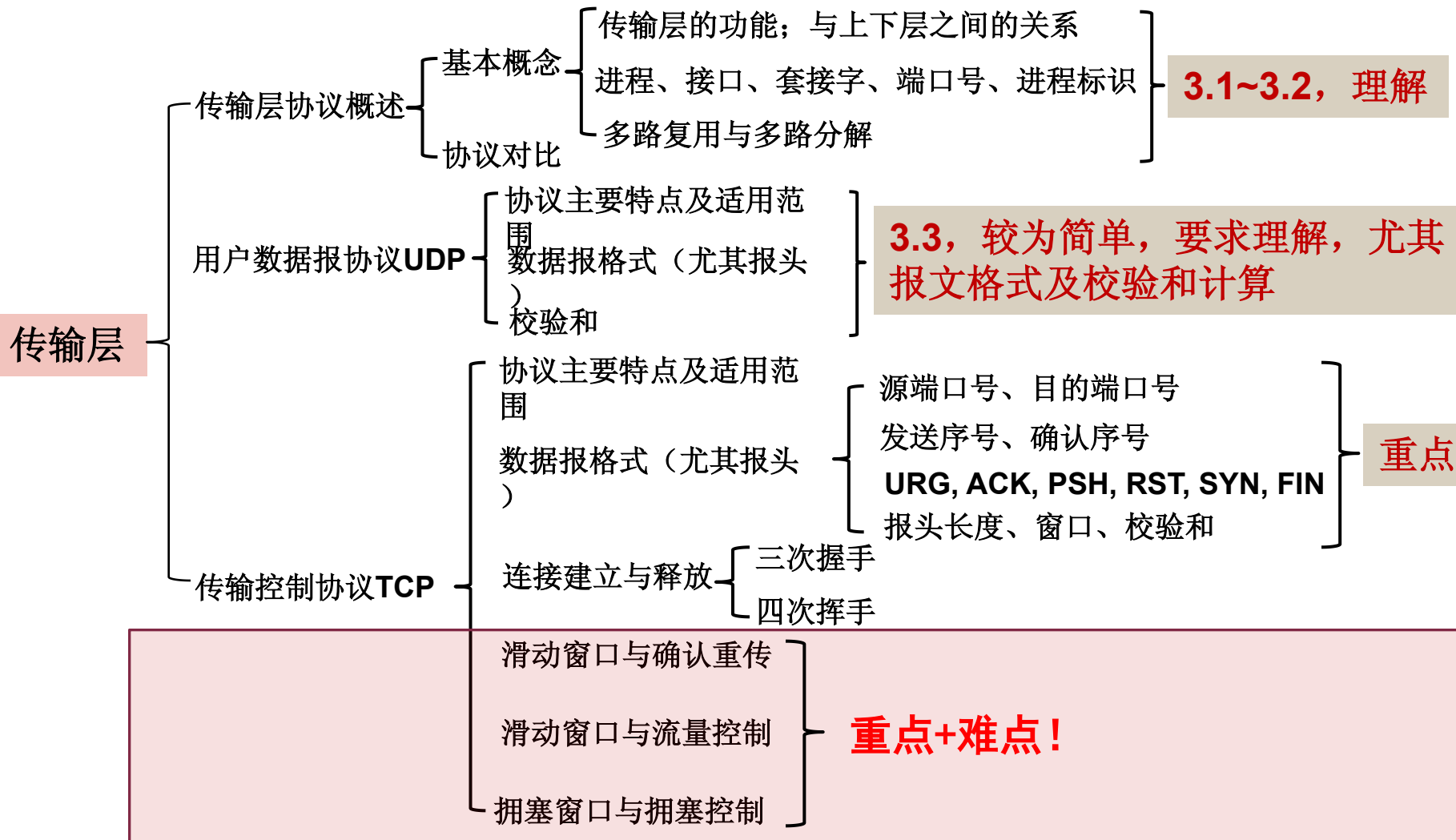
3.4 传输控制协议

■ 3.4.3 TCP基本通信过程-拓展思考题

1. 为什么TCP释放连接用了4个报文段，而TCP建立连接只需要3个？
2. 对于数据字段长度=0的SYN/FIN报文段S，假定S的序号为x，为什么其ACK报文段的序号等于x+1？
3. 请查阅TCP状态机图示，了解客户机和服务器在TCP通信过程中的状态变化



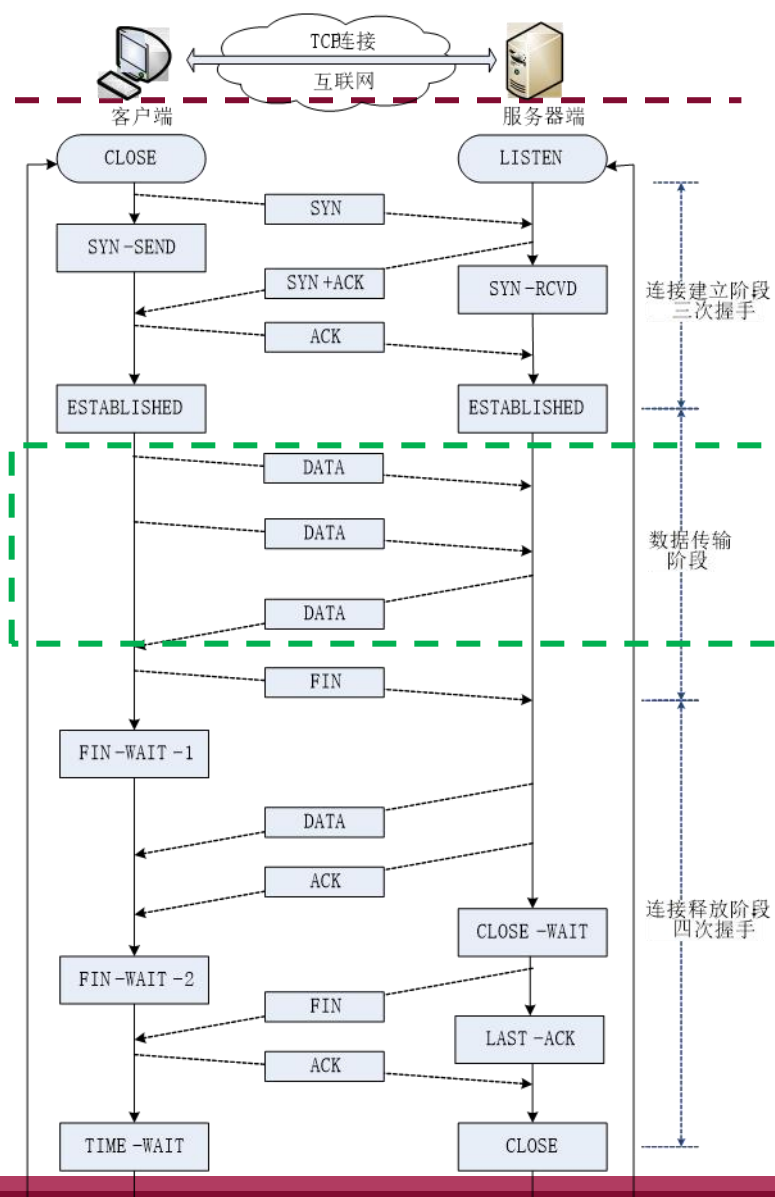
本讲概述



回顾-TCP通信过程

■ 数据双向传输阶段

- 以**滑动窗口**中的多个报文段为单位的双向传输
- 应用进程将数据以**字节流**发送，无需考虑发送数据字节长度，由TCP负责将字节流分段打包
- 借助滑动窗口，实现按序的，无差错、不丢失、不重复传送字节流
- 提供差错控制功能，保证正确接收字节流（通过**差错检测、确认、重传**实现）



3.4 传输控制协议

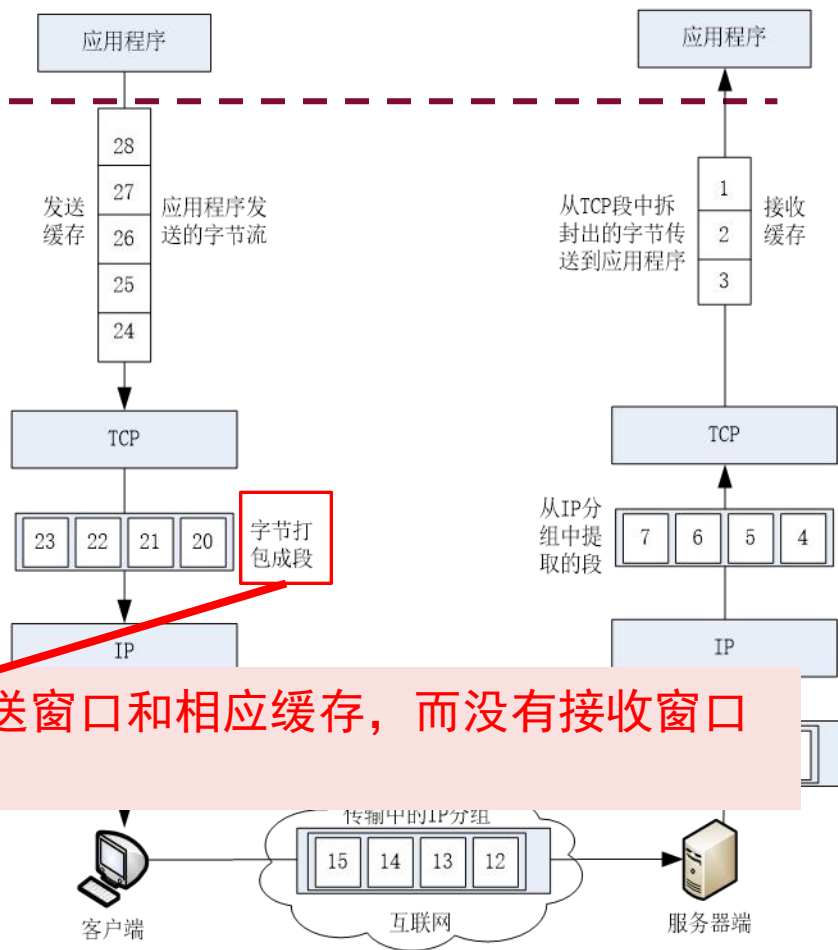
3.4.4 TCP滑动窗口与确认重传机制

- 2个缓存、2个窗口（控制字节流传输过程）

- 发送方缓存：用于存储准备发送的数据
- 发送窗口：窗口值不为0，可以发送报文段
- 接收方缓存：将正确接收的字节流写入缓存等待接收应用程序

思考：在一个通信进程中，发送方是否只有发送窗口和相应缓存，而没有接收窗口和相应缓存？为什么？

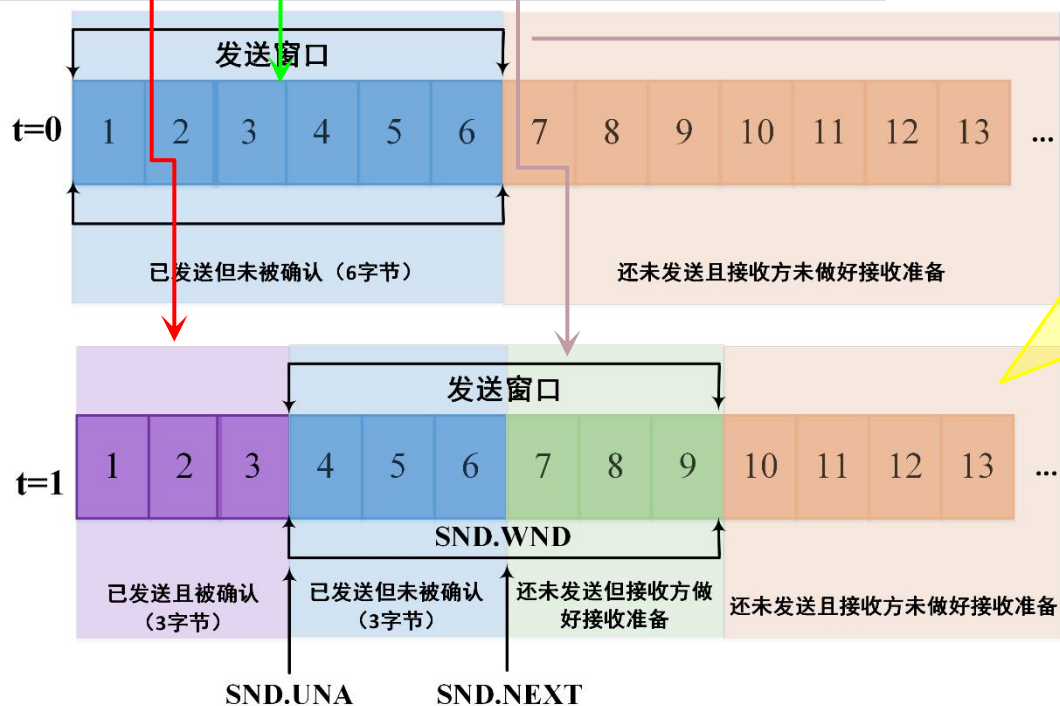
- 字节流分段，按段（序号）传输，捎带确认（确认号）
- 通过滑动窗口跟踪、记录发送状态，实现差错控制



3.4 传输控制协议

3.4.4 滑动窗口与确认重传机制-示例

假定进程A与B建立了一个TCP连接，A向B发送了的报文段覆盖字节编号[1,6]。A收到了B发来的ACK报文段，其中确认号是4，窗口大小是6字节。



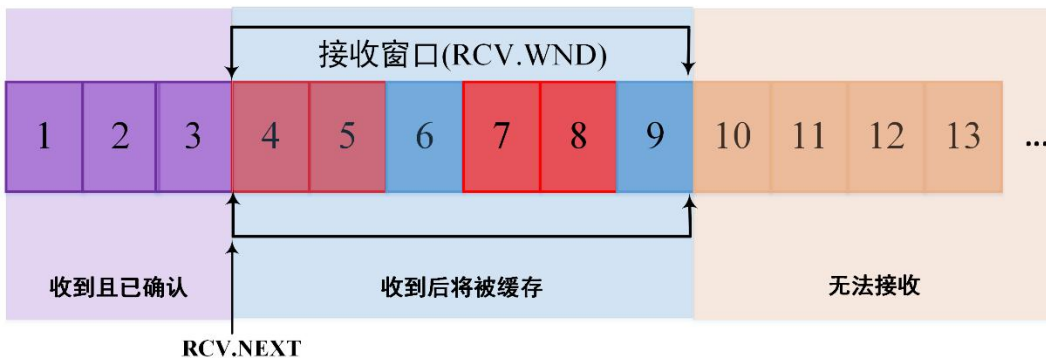
发送方A还可以发送的字节长度
= $\text{SND.UNA} + \text{SND.WND} - \text{SND.NXT}$
= $4 + 6 - 7$
= 3

3.4 传输控制协议

3.4.4 滑动窗口与确认重传机制

假定进程A与B建立了一个TCP连接，A向B发送了的报文段覆盖字节编号[1,6]。A收到了B发来的ACK报文段，其中**确认号**是**4**，**窗口大小**是**6**字节。

B的接收窗口的一种可能形式：



- 1) 进程B期望接收的字节在范围[4,9]之内
- 2) 若收到的字节的序号 ≤ 3 ，则是**重复**的数据，**丢弃**
- 3) 若收到的字节的序号 ≥ 10 ，则是**跳序**的数据，**丢弃**
- 4) 只有当收到的字节序号距离RCV.NXT指针是**连续的**(如收到序号为4,5的两个字节)，接收窗口才会向右滑动(如 $RCV.NXT += 2$)
- 5) TCP支持SACK选项，即可以对[4,9]之间的**非连续字节**进行选择确认。如收到序号4,5,7,8的四个字节，B向A发送ACK报文段，其中确认号是**6**，TCP选项为[7,8]，表示序号7到8之间的字节被收到。

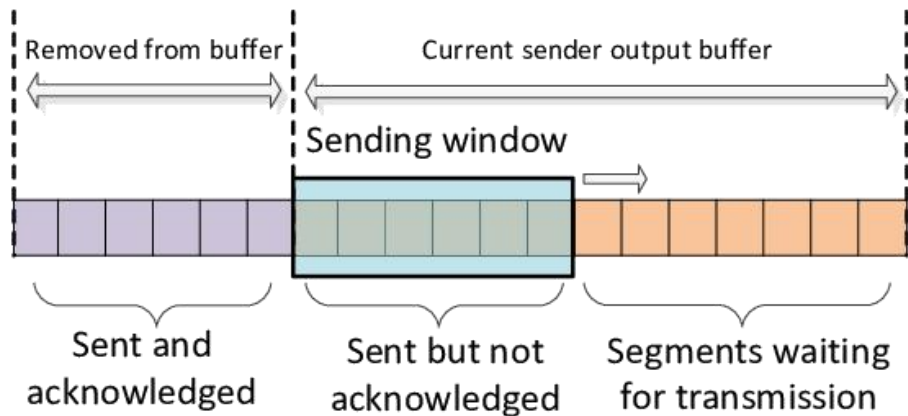


3.4 传输控制协议

3.4.4 TCP滑动窗口与确认重传机制

TCP滑动窗口协议的特点：

- 使用发送和接受缓冲区，以及滑动窗口机制控制TCP连接上的字节传输
- TCP滑动窗口面向字节流，可以起到差错控制和流量控制作用
- 接收方可以在任何时候发送确认，窗口大小可由接收方根据需要增大或减少
- 发送窗口值不能超过接收窗口值，发送方可以根据自身需要来决定



3.4 传输控制协议

3.4.4 TCP滑动窗口与确认重传机制



- 回退N步 GBN

- 假设丢失了第2个报文段，不管之后的报文段是否已正确接收，从第2个报文段的第1个字节序号151开始，重发所有的4个报文段。

- 选择重发 SR

- 接收方收到不连续的字节时，如果这些字节的序号都在接收窗口之内，则首先接收缓存这些字节，并将丢失的字节流序号通知发送方，发送方只需重发丢失的报文段，而不需要重发已经接收的报文段。

3.4 传输控制协议

- 3.4.4 TCP滑动窗口与确认重传机制
- TCP使用GBN还是SR?

Go-Back-N

➤接收方:

- 使用累积确认
- 不缓存失序的分组
- 对失序分组发送重复ACK

➤发送方:

- 超时报重传从基序号开始的所有分组

TCP

➤接收方:

- 使用累积确认
- 缓存失序的报文段
- 对失序报文段发送重复ACK
- 增加了推迟确认

➤发送方:

- 超时报重传最早未确认的报文段
- 增加了快速重传



3.4 传输控制协议

- 3.4.4 TCP滑动窗口与确认重传机制
- TCP使用GBN还是SR?

SR

➤接收方:

- 缓存失序的分组
- 单独确认每个正确收到的分组

➤发送方:

- 每个分组使用一个定时器
- 仅重传未被确认的分组

修改的TCP [RFC2018]

➤接收方:

- 缓存失序的报文段
- SACK选项头中给出非连续数据块的上下边界

➤发送方:

- 只对最早未确认的报文段使用一个定时器
- 仅重传接收方缺失的数据
- 增加了快速重传



3.4 传输控制协议

- 3.4.4 TCP滑动窗口与确认重传机制
- TCP结合了GBN和SR的优点
- TCP的差错恢复机制可以看成是GBN和SR的混合体：
 - 定时器的使用：与GBN类似，只对最早未确认的报文段使用一个定时器
 - 超时重传：与SR类似，只重传缺失的数据
- TCP在减小定时器开销和重传开销方面要优于GBN 和 SR！



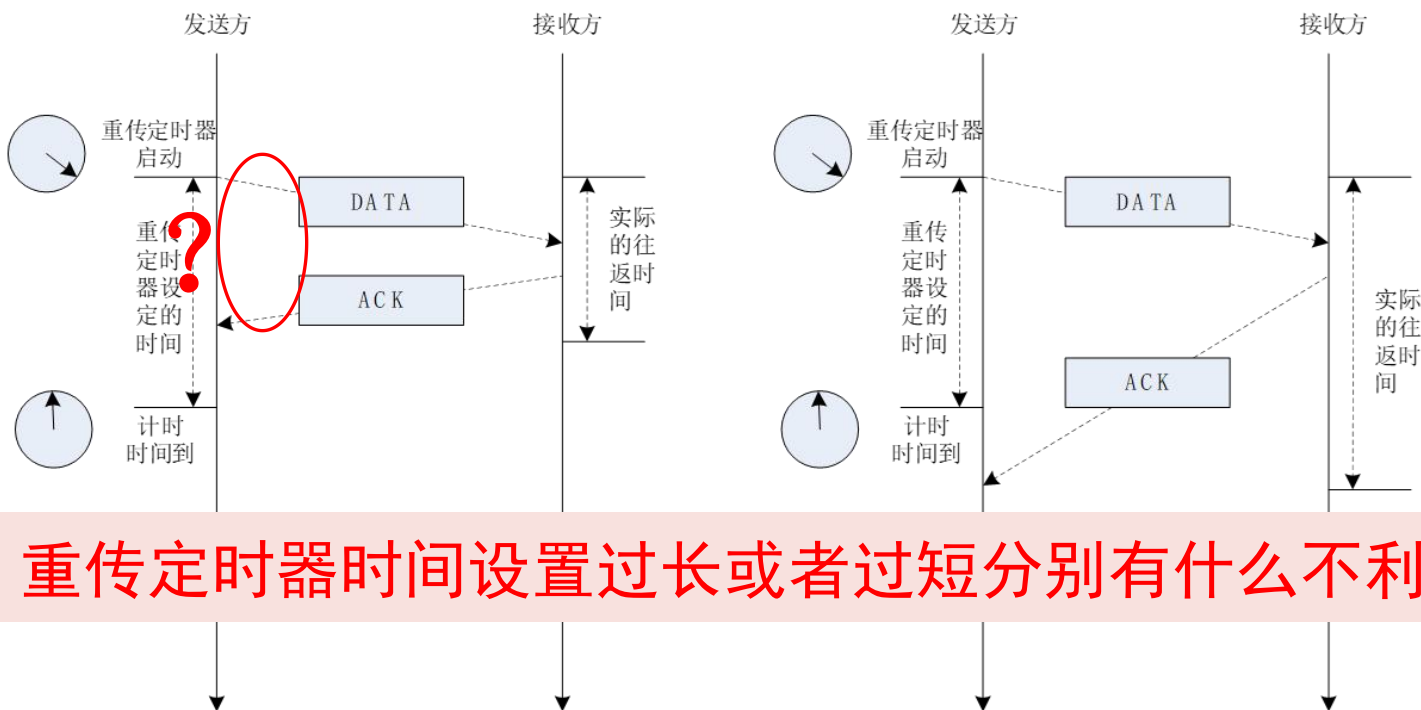
3.4 传输控制协议

考虑如果发送方一直没有收到接收方发来的确认，怎么办？

3.4.4 TCP滑动窗口与超时重传机制

重传定时器

- 处理报文确认与等待重传的时间。发送一个报文，将其副本放入重传队列

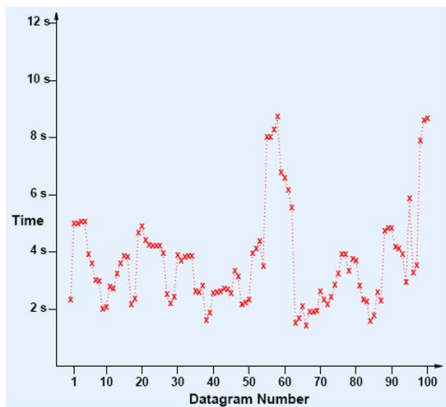


思考：重传定时器时间设置过长或者过短分别有什么不利影响？

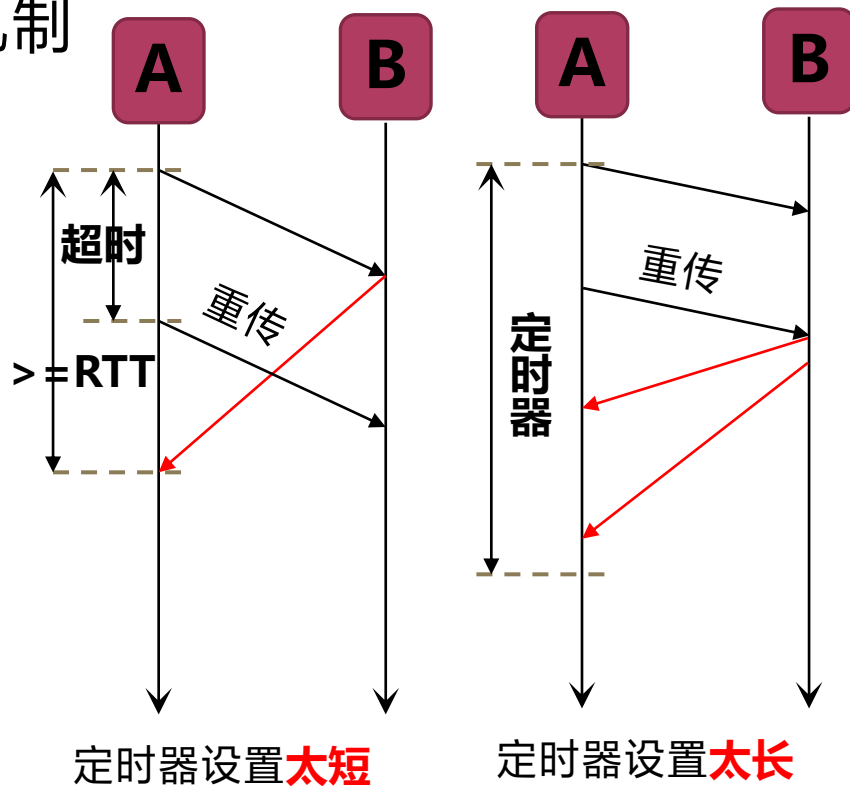
3.4 传输控制协议

3.4.4 TCP滑动窗口与超时重传机制

- 如果把超时重传时间设置得**太短**，就会引起很多报文段**不必要的重传**，使网络负荷增大；
- 但若把超时重传时间设置得**过长**，则又使网络的**空闲时间增大**，降低了传输效率。
- 定时器设置与报文段往返时间**RTT**紧密相关。RTT表示一个报文段自发出到收到ACK的时间间隔。



因特网上相继传输
100个IP数据报的
往返时间测量值



- RTT随网络状态而**随机**波动，既非定值，也无规律
- 发送数据报的**同时**，需要对该报文的RTT进行估计

3.4 传输控制协议

■ 3.4.4 TCP滑动窗口与超时重传机制

$$\text{Timeout} = \beta \times \text{RTT}$$

■ 其中

- Timeout: 本次定时器的时间值
- RTT: 对本次发出报文段的往返时间的估计值
- β : 常数值, 大于1, 推荐设置为2

$$\text{RTT} = \alpha \times \text{旧RTT} + (1 - \alpha) \times \text{最新测量RTT}$$

■ 其中

- α : 常数值, 小于1, 推荐设置为0.125

示例: 请参见教材例子并认真加以理解! (指数加权平均)



3.4 传输控制协议

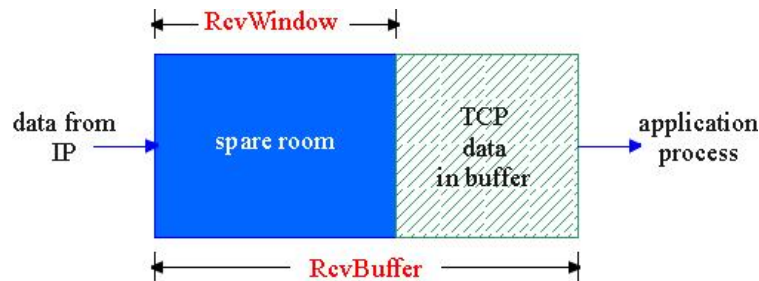
3.4.5 TCP窗口与流量控制、拥塞控制

• 流量控制

- 由发送方控制发送速率，使之不超过接收速率，防止接收方来不及接收字节流，而出现报文丢失现象

• 基本过程：

- 接收方从缓存中读取速度大于等于字节到达速度，接收方在每个确认中发出一个**非零窗口通告**
- 如果发送方发送速度比接收方读取速度快，将造成缓冲区被全部占用，之后到达的字节因缓冲区溢出而丢弃。此时，接收方必须发出一个“**零窗口**”的通告。告知当发送方停止发送（直到接收“非零窗口”通告为止）。
- 接收方需要接收能力给出一个合适的接收窗口，并将它写入TCP报头中，通知发送方。接收窗口又称为**通知窗口**。



3.4 传输控制协议

■ 3.4.5 TCP窗口与流量控制、拥塞控制

- 触发零窗口事件的流量控制基本事件逻辑：

1. 零窗口(Zero window)通告

- 当TCP接收方的缓冲区满(如应用层未能及时提取数据), 则接收方往发送方发送**纯ACK报文段**(数据长度=0, 窗口字段=0), 称之为**零窗口通告**

2. 窗口更新(Window update)

- 当接收方缓冲区从满状态变到有可用空间时, 接收方向发送方发送纯ACK报文段(数据长度=0, 窗口字段>0), 称之为**窗口更新**

3. 窗口探测(Window probe)

- 因为纯ACK报文段(数据长度=0, ACK=1的报文段)不被TCP可靠递送, 则第2步的窗口更新有可能在网络中丢失, 导致收发双方都处于等待的死锁状态。
- 为避免活锁, 发送方使用一种叫**坚持定时器**(persist timer)来定期触发
 - 发送方往接收方发送窗口**探测报文段**(数据长度>0, 保证被TCP递送)。作为响应, 接收方将自己的缓冲区可用空间大小放入ACK报文段的窗口字段, 由此, 发送方获知接收方是否能继续接收数据。



3.4 传输控制协议

3.4.5 TCP窗口与流量控制

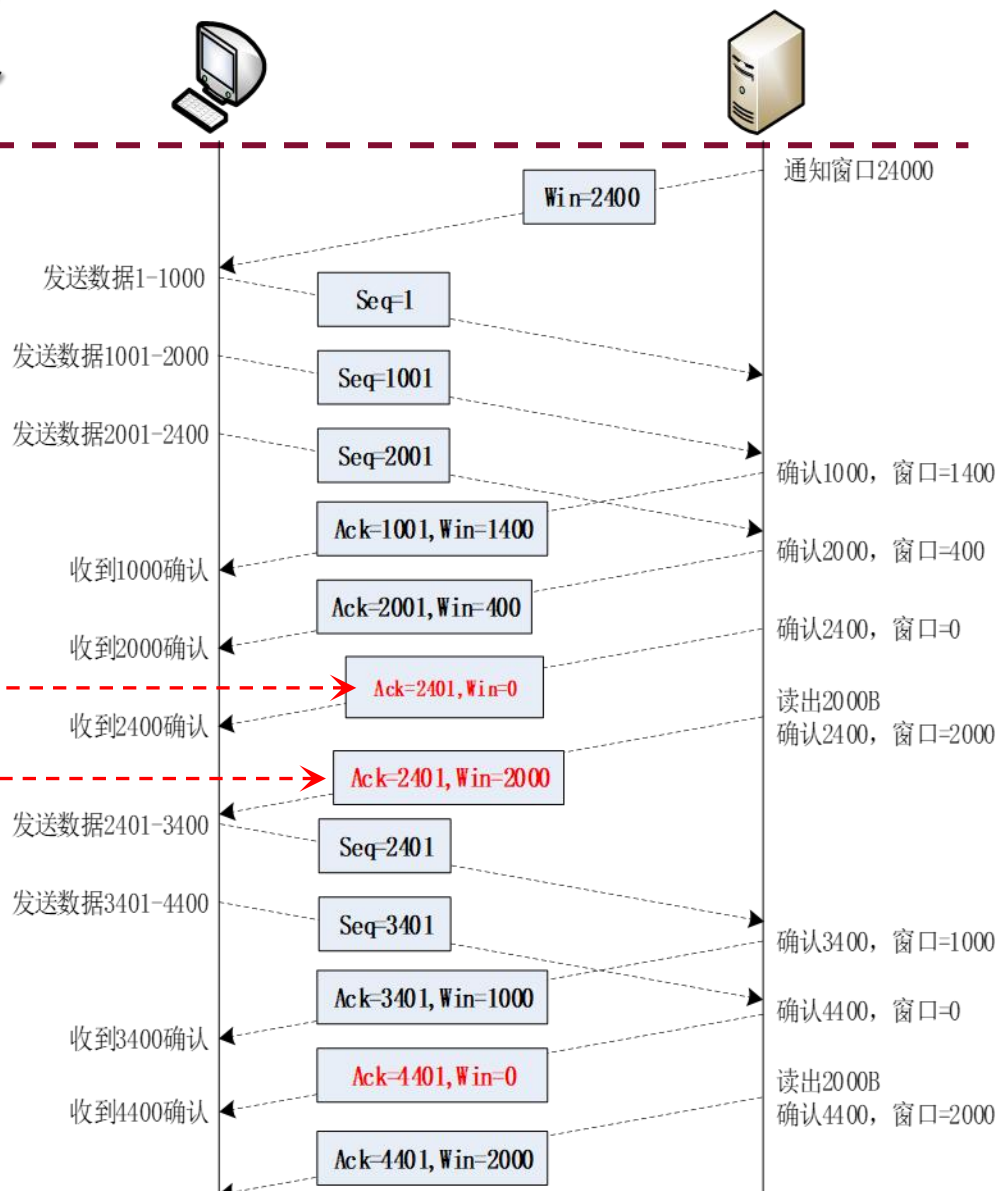
- 触发零窗口事件的流量控制：

零窗口通告：纯ACK报文段，数据长度=0，窗口字段=0

窗口更新：纯ACK报文段，数据长度=0，窗口字段=0

考虑该报文易丢失！

窗口探测：非纯ACK报文段，数据长度=1，ACK=1，发送方发送



3.4 传输控制协议

3.4.5 TCP窗口与流量控制、拥塞控制

触发零窗口事件的流量控制：实例

No.	Time	Source	Destination	Info
150	0.1314	10.0.1.37	10.0.1.33	6666 > 53005 [ACK] Seq=1 Ack=131497 win=327 Len=0
151	4.7692	10.0.1.33	10.0.1.37	[TCP window Full] 53005 > 6666 [ACK] Seq=131497 Ack=1 Win=131070 Len=327
152	4.9792	10.0.1.37	10.0.1.33	[TCP zerowindow] 6666 > 53005 [ACK] Seq=1 Ack=131824 win=0 Len=0
153	9.7706	10.0.1.33	10.0.1.37	[TCP zerowindowProbe] 53005 > 6666 [ACK] Seq=131824 Ack=1 win=131070 Len=1
154	9.7716	10.0.1.37	10.0.1.33	[TCP zerowindowProbeAck] [TCP zerowindow] 6666 > 53005 [ACK] Seq=1 Ack=131824 win=0
155	14.772	10.0.1.33	10.0.1.37	[TCP zerowindowProbe] 53005 > 6666 [ACK] Seq=131824 Ack=1 win=131070 Len=1
156	14.773	10.0.1.37	10.0.1.33	[TCP zerowindowProbeAck] [TCP zerowindow] 6666 > 53005 [ACK] Seq=1 Ack=131824 win=0
157	19.773	10.0.1.33	10.0.1.37	[TCP zerowindowProbe] 53005 > 6666 [ACK] Seq=131824 Ack=1 win=131070 Len=1
158	19.774	10.0.1.37	10.0.1.33	[TCP zerowindowProbeAck] [TCP zerowindow] 6666 > 53005 [ACK] Seq=1 Ack=131824 win=0
159	20.143	10.0.1.37	10.0.1.33	[TCP window update] 6666 > 53005 [ACK] Seq=1 Ack=131824 win=3752 Len=0
160	20.143	10.0.1.37	10.0.1.33	[TCP window update] 6666 > 53005 [ACK] Seq=1 Ack=131824 win=65535 Len=0
161	20.143	10.0.1.33	10.0.1.37	53005 > 6666 [ACK] Seq=131824 Ack=1 win=131070 Len=1448
162	20.143	10.0.1.33	10.0.1.37	53005 > 6666 [ACK] Seq=133272 Ack=1 win=131070 Len=1448
163	20.143	10.0.1.33	10.0.1.37	53005 > 6666 [ACK] Seq=134720 Ack=1 win=131070 Len=1448

在零窗口(zerowindow)通告之后，每隔5秒，有一个窗口探测事件(ZerowindowProbe, No.153,155,157)由发送方主动发出。在接收方的应用层提取出TCP数据后，接收方发出窗口更新(window update, No.159, No.160)



3.4 传输控制协议

■ 3.4.5 TCP窗口与流量控制、拥塞控制

• 糊涂窗口综合症 (Silly Window Syndrome, SWS)

- 定义：当数据的发送速度很快、而消费速度很慢时，接收方不断发送微小窗口通告，发送方不断发送很小的数据分组，小的报文段在TCP连接上传输，导致有效数据的通信效率低下，大量带宽被浪费。小报文段的**数据字段长度**远小于报文段首部长度20+IP首部长度20
- SWS可由TCP连接的任何一端引起：接收方通过**纯ACK报文段**发送小的窗口通告导致SWS；发送方过于积极传输缓冲区中的剩余数据导致SWS
- 解决方案：
 - 接收方启发式策略
 - 使用TCP延迟确认（Delayed ACK）来减少频繁的确认证包发送。
 - Clark算法等
 - 发送方启发式策略
 - Nagle算法等



3.4 传输控制协议

■ 3.4.5 TCP窗口与流量控制、拥塞控制

- 接收端避免糊涂窗口综合症的策略：

- 通告零窗口之后，不发送小的窗口通告。仅当窗口大小显著增加之后才发送更新的窗口通告
- 什么是显著增加：窗口大小达到缓存空间的一半或者一个MSS，取两者的较小值（Clark算法）

- TCP执行该策略的做法：

- 当窗口大小不满足以上策略时，推迟发送确认（但最多推迟500ms，且至少每隔一个报文段使用正常方式进行确认），寄希望于推迟间隔内有更多数据被消费
- 仅当窗口大小满足以上策略时，才通告新的窗口大小



3.4 传输控制协议

3.4.5 TCP窗口与流量控制、 拥塞控制

- 发送方避免糊涂窗口综合症的策略：
 - 发送方应积聚足够多的数据再发送，以防止发送太短的报文段
- 问题：发送方应等待多长时间？
 - 若等待时间不够，报文段会太短
 - 若等待时间过久，应用程序的时延会太长
 - 更重要的是，TCP不知道应用程序不会在最近的将来生成更多的数据

- Nagle算法的解决方法：

- 在新建连接上，当应用数据到来时，组成一个TCP段发送（那怕只有一个字节）。在收到确认之前，后续到来的数据放在发送缓存中。
- 一般不发送小报文段，但在下述情况，发送方立即发送数据：1）当发送缓冲区的数据大到可以组装成一个MSS；2）当发送缓冲区的数据大到接收方所有通知窗口中最大窗口尺寸的一半；3）发送方不在等待ACK、或者Nagle算法被禁止。



3.4 传输控制协议

■ 3.4.5 TCP窗口与流量控制、拥塞控制-拓展思考题

1. 为什么TCP不能可靠递送纯ACK报文段？
2. 仔细观察本节“1.触发零窗口事件的流量控制~示例”，假设零窗口探测(zerowindowProbe)报文段P的ACK报文段为Q，为什么 $Q.ack \neq P.Seq + P.Len$ ？
3. Nagle算法与TCP delayed acknowledgments有冲突，试查阅互联网解释其中原因。



3.4 传输控制协议

3.4.5 TCP窗口与流量控制、拥塞控制

内容提纲:

1. 拥塞现象
2. 拥塞控制
3. 拥塞事件的判断方法
4. TCP拥塞控制算法
 - 4.1 慢开始
 - 4.2 拥塞避免
 - 4.3 快重传
 - 4.4 快恢复
5. 问题

理解两张图



3.4 传输控制协议

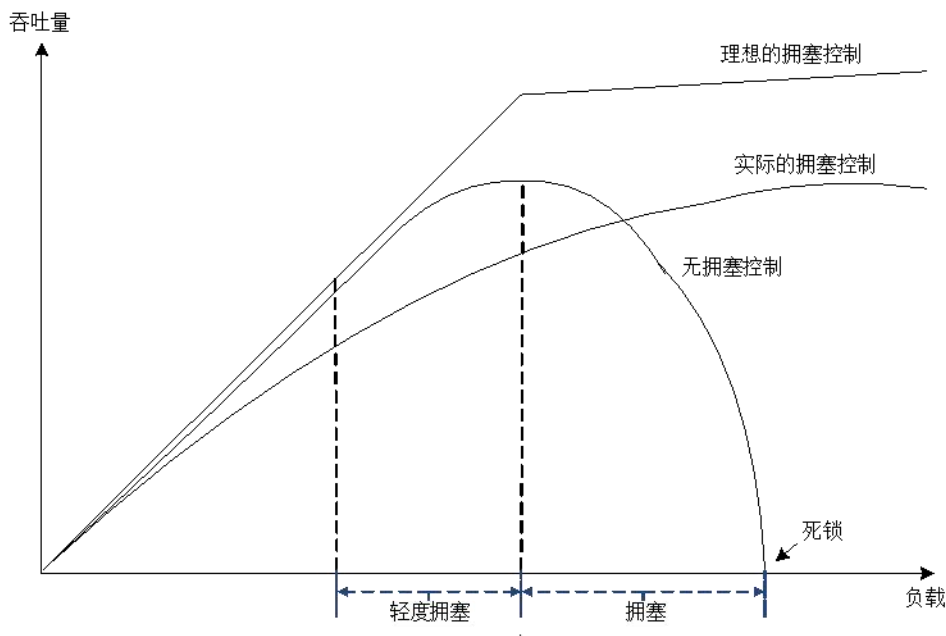
3.4.5 TCP窗口与流量控制、 拥塞控制

■ 拥塞现象

- 当多个主机端点通过TCP向整个网络注入数据时，TCP如果不对**注入数据的速率**进行控制，将导致整个网络出现拥堵：
- 往返时延增大导致**TCP重传**，进一步增加拥堵
- 中间路由器的缓冲容量有限。当IP数据报的到达率持续超过发出率时，中间路由器的缓冲变满，只好丢弃新进的数据包，导致**TCP重传**。

需要在TCP协议中增加拥塞控制，**及时感知**网络的**拥堵和顺畅**情况，**自适应地减少和增加**向网络注入数据的速率

TCP感知网络拥塞：丢包计数、度量往返时延、显示拥塞通知



大量网络资源用于：

- (1) 重传丢失的分组
- (2) (不必要地) 重传延迟过大的分组
- (3) 转发最终被丢弃的分组

3.4 传输控制协议

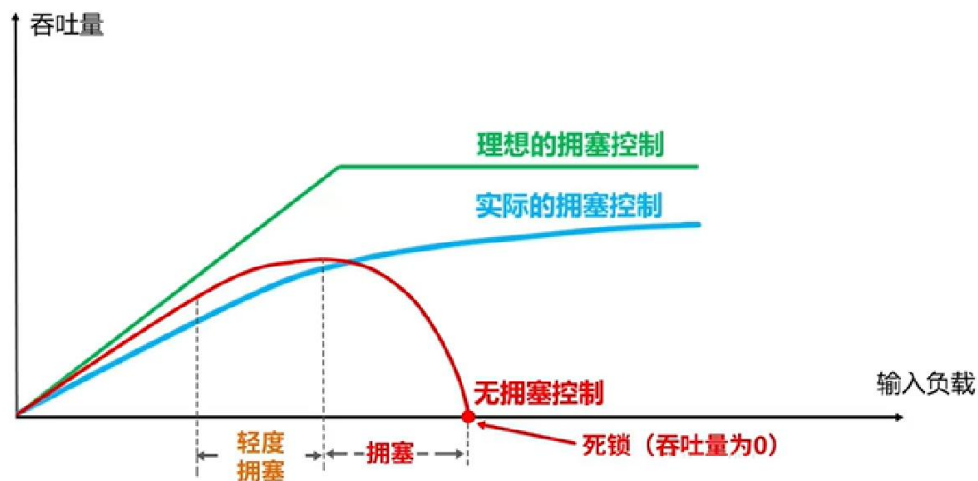
3.4.5 TCP窗口与流量控制、拥塞控制

- TCP使用端到端拥塞控制机制：

- 发送方根据自己感知的网络拥塞程度，限制其发送速率

- 需要回答三个问题：

- 发送方采用什么机制来限制发送速率？
- 发送方如何感知网络拥塞？
- 发送方感知到网络拥塞后，采取什么策略调节发送速率？



3.4 传输控制协议

■ 3.4.5 TCP窗口与流量控制、拥塞控制

■ 拥塞控制

- 实现拥塞控制最基本手段：TCP滑动窗口技术。
- 发送数据，既要考虑接收能力，又要避免网络发生拥塞
- 发送窗口计算

发送窗口 = $\min(\text{通知窗口}, \text{拥塞窗口})$

- **通知窗口** $rwnd$ ：接收方允许接收的能力，来自接收方流量控制（将“通知窗口”值放在TCP报头中，传送给发送端）。
- **拥塞窗口** $cwnd$ ：发送方根据网络拥塞情况得出的窗口值，来自发送方的流量控制。



3.4 传输控制协议

■ 3.4.5 TCP窗口与流量控制、拥塞控制

■ 拥塞判定

- 重传定时器超时

- 现在通信线路的传输质量一般都很好，因传输出差错而丢弃分组的概率是很小的（远小于 1 %）。只要出现了超时，就可以猜想网络可能出现了拥塞

- 收到三个相同（重复）的 ACK

- 个别报文段会在网络中丢失，预示可能会出现拥塞（实际未发生拥塞），因此可以尽快采取控制措施，避免拥塞

- *选择性确认（SACK, Selective Acknowledgment）

- 一种扩展机制，接收方明确告知发送方哪些数据包已经成功接收，哪些尚未接收

- 重复累积确认

- 通过分析累积确认的变化来推断数据包丢失。例如，如果一个数据包之后的所有数据都被确认，可推断该数据包已丢失

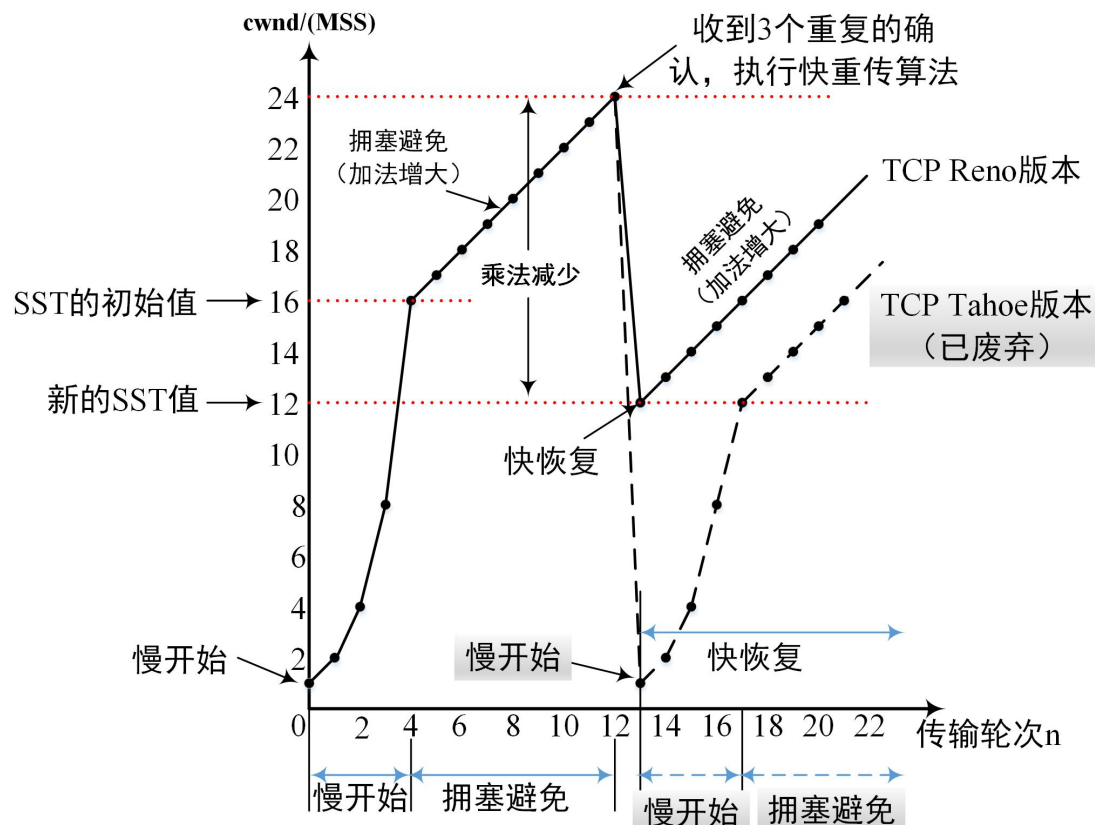


3.4 传输控制协议

3.4.5 TCP窗口与流量控制、拥塞控制

拥塞控制

- 慢开始
- 拥塞避免
- 快重传
- 快恢复



3.4 传输控制协议

3.4.5 TCP窗口与流量控制、拥塞控制

慢开始

- 慢开始阈值SST

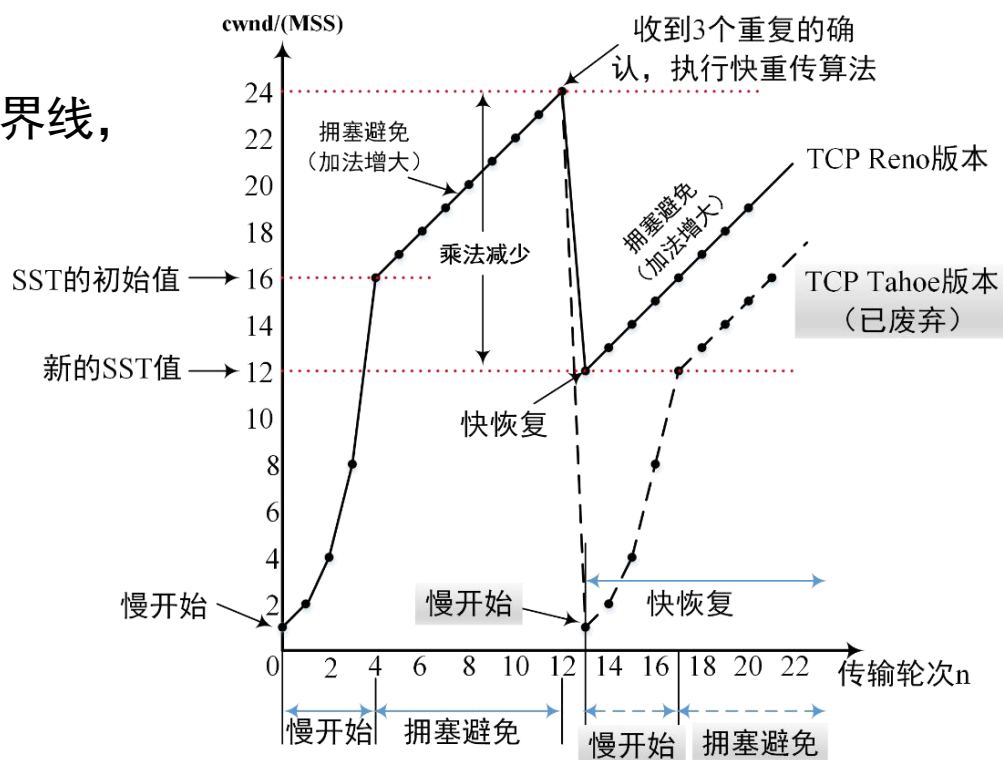
- 慢开始阶段和拥塞避免阶段的分界线，初始值由算法设定

- 慢开始阶段

- $cwnd \leq SST$
- 在该阶段，cwnd初始值为1，且每结束一个RTT，cwnd翻倍

- 当cwnd=SST时

- 进入拥塞避免阶段

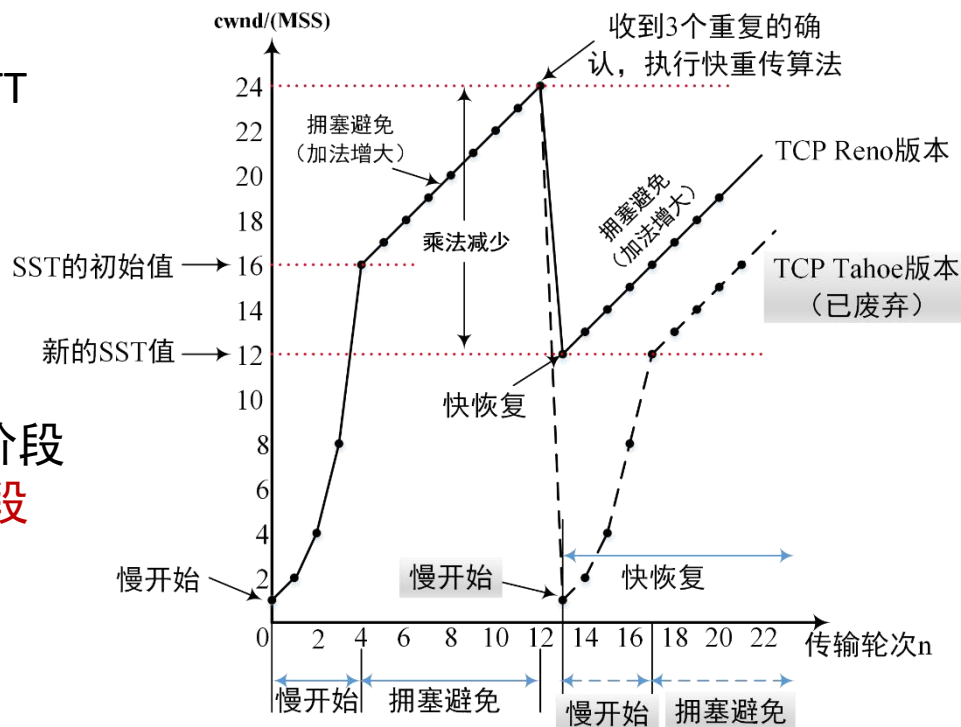


3.4 传输控制协议

3.4.5 TCP窗口与流量控制、拥塞控制

■ 拥塞避免

- 当 $cwnd \geq SST$ 时
 - 进入拥塞避免阶段。每完成一个RTT $cwnd$ 加1, 随传输轮次 n 线性增长
- 当发生拥塞时
 1. 拥塞避免阶段结束
 2. 设置 $SST = cwnd/2$
 3. $cwnd$ 重为1, 进入新的慢开始阶段或 $cwnd$ 减半, 进入快恢复阶段
- 拥塞判定
 - 定时器超时或者重复确认

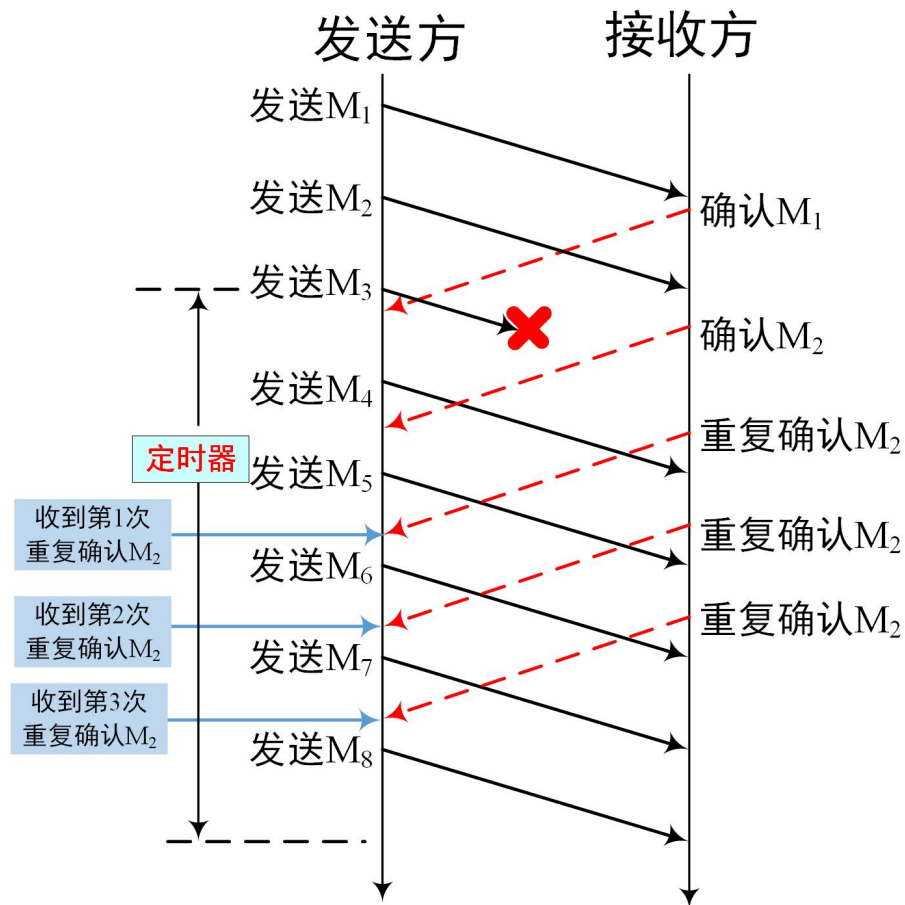


3.4 传输控制协议

3.4.5 TCP窗口与流量控制、拥塞控制

快重传

- “快重传”规定：接收方应及时向发送方连续3次发出对M2的“重复确认”，要求尽早重传未被确认的报文
- 当发送方接收到重复的ACK时，立即重传丢失的报文段M3，而不必等到M3的重传定时器超时才重传
- TCP Tahoe版和TCP Reno版都支持快重传

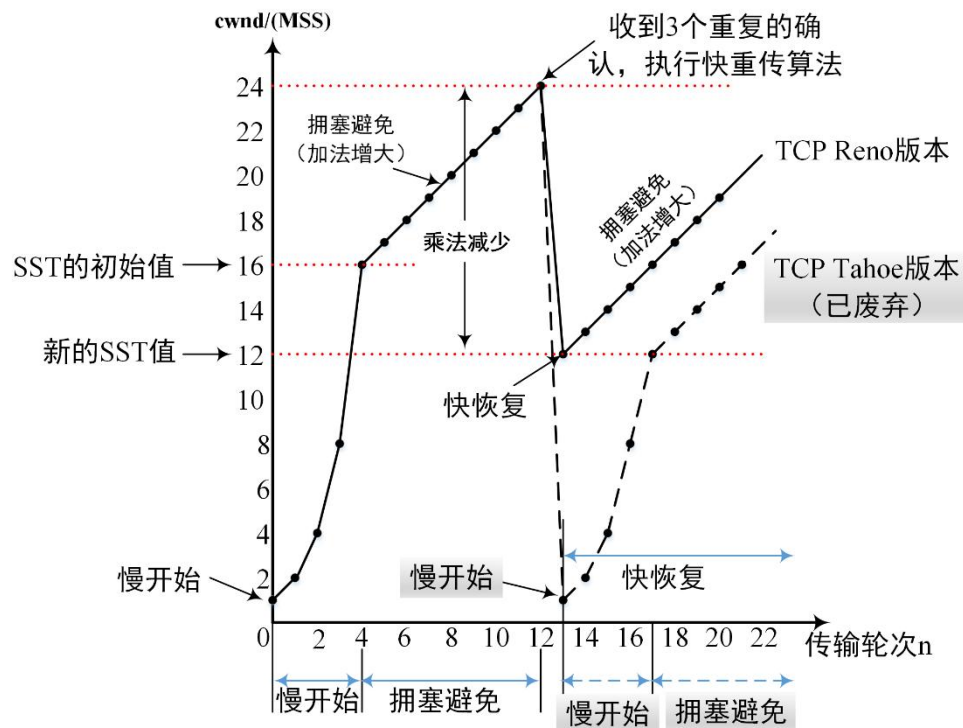


3.4 传输控制协议

3.4.5 TCP窗口与流量控制、拥塞控制

快恢复

- TCP Reno版在发生拥塞事件时，将cwnd减半，直接进入拥塞避免阶段

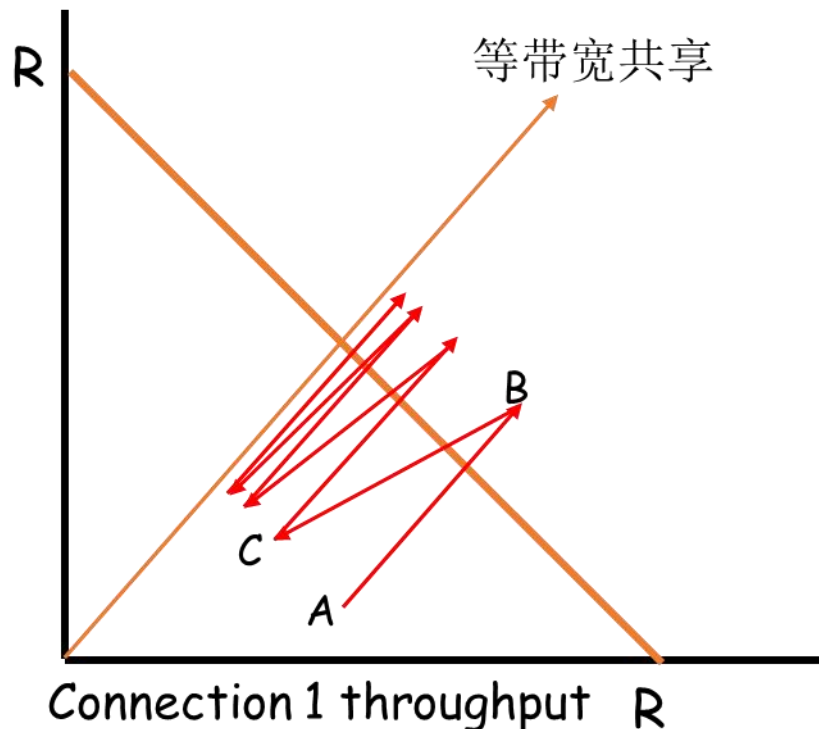


3.4 传输控制协议

3.4.5 TCP窗口与流量控制、拥塞控制

为什么TCP是公平的

- 考虑两条竞争的连接（各种参数相同）
共享带宽为 R 的链路：
- 加性增：连接1和连接2按照相同的速率增大各自的拥塞窗口，得到斜率为1的直线
- 乘性减：连接1和连接2将各自的拥塞窗口减半



3.4 传输控制协议

- 3.4.5 TCP窗口与流量控制、**拥塞控制**
- TCP公平性更复杂的情形
 - 若相互竞争的TCP连接具有不同的参数（RTT、MSS等），不能保证公平性
 - 若应用（如web）可以建立多条并行TCP连接，不能保证带宽在应用之间公平分配，比如：
 - 一条速率为 R 的链路上有9条连接
 - 若新应用建立一条TCP连接，获得速率 $R/10$
 - 若新应用建立11条TCP，可以获得速率超过 $R/2$!



3.4 传输控制协议

■ 3.4.5 TCP窗口与流量控制、 拥塞控制

■ 显式拥塞通知（ECN）

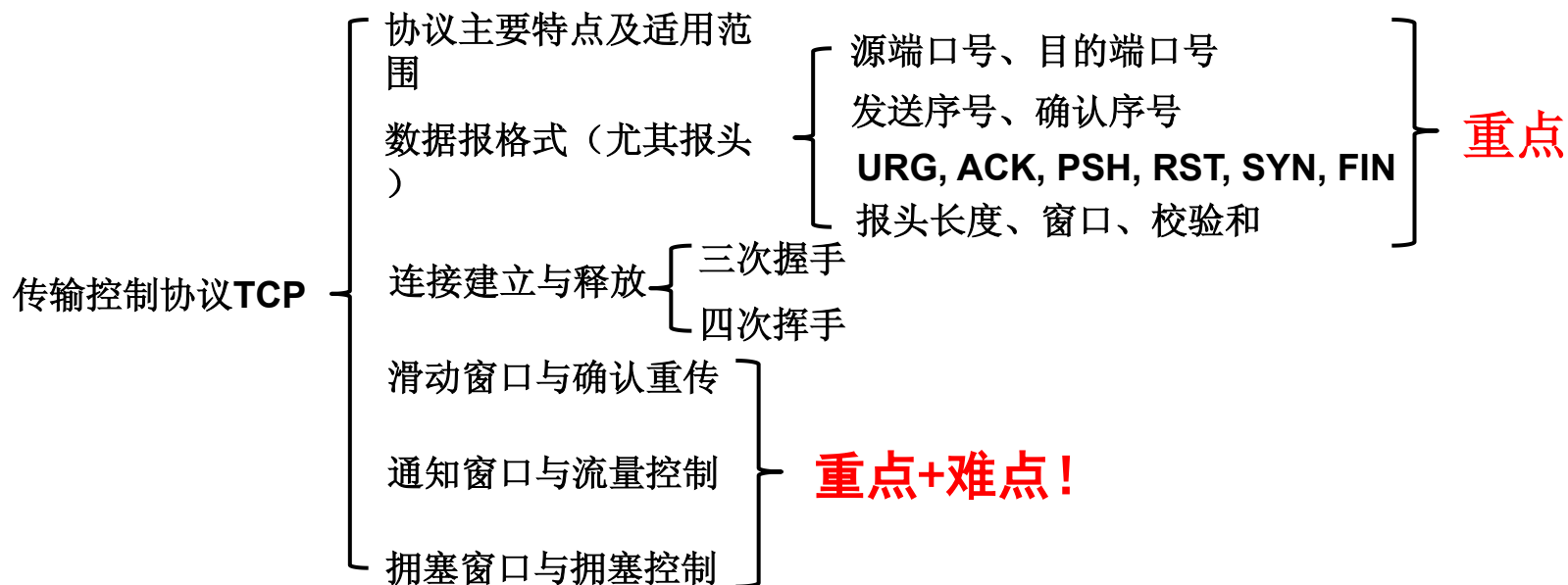
- 定义于RFC 3168
- ECN允许拥塞控制的端对端通知而避免丢包。ECN为一项可选功能，如果底层网络设施支持，则可能被启用ECN的两个端点使用。
 - TCP/IP网络一般通过丢弃数据包来表明信道阻塞。

■ ECN的工作原理

- 在IP头部中的ECN字段标志位（ECT）用于指示数据包的发送方和接收方都支持ECN。当路由器检测到拥塞时，会将该数据包的ECN字段设置为CE标志位，表示网络中发生了拥塞。
- 接收方收到标记为CE的数据包后，会将相应的信息传达给发送方，通知发送方网络中发生了拥塞。
- 发送方收到拥塞通知后，可以采取一些措施，如减少发送速率、执行拥塞避免算法等，以避免继续加重网络拥塞



TCP总结



发送窗口 = $\text{Min}(\text{通知窗口}, \text{拥塞窗口})$

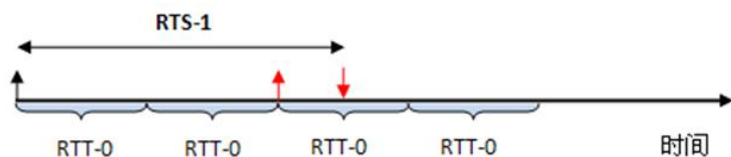
滑动窗口 — 发送窗口和接收窗口滑动技术

通知窗口 — 接收窗口

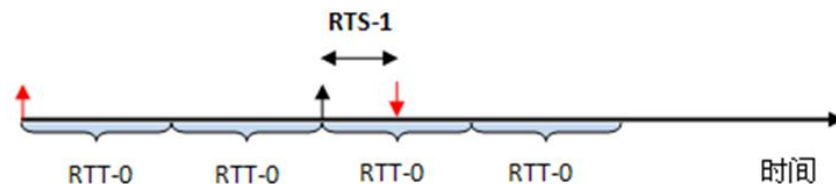


3.4 传输控制协议

- 准确测量往返时间样本的问题（拓展内容，不做要求）
 - TCP协议并没有定义重传的报文段与原先的报文段的差别，故无法确定将一个确认(ACK)与第几个重传关联，即TCP协议存在确认二义性
 1. 若将确认与原先的报文段关联：将导致 R_{ts} 较真实值大， R_{tt} 较真实值大；在每个发送的报文段至少丢失一次的网络中，将导致 R_{tt} 无限增大
 2. 若将确认与最近重传的报文段关联：将导致 R_{ts} 较真实值小，导致 R_{tt} 较真实值小；由于密集重发报文段，导致 R_{tt} 越来越小；最后 R_{tt} 会稳定在某个值 x ，而真实的往返时间大约是 x 的整数倍



对应第1种情况



对应第2种情况

$\beta=2$ ，红色箭头表示真实的一对收发

3.4 传输控制协议

■ 准确测量往返时间样本的问题

- 简单忽略重传报文段的问题：

- 重传意味着超时值可能偏小了，需要增大
- 若简单忽略重传报文段（不更新EstimatedRTT），则超时值也不会更新，超时设置过小的问题没有解决

- 解决方法：

- 采用定时器补偿策略，发送方每重传一个报文段，就直接将超时值增大一倍（不依赖于RTT的更新）
- 若连续发生超时事件，超时值呈指数增长（至一个设定的上限值）



3.4 传输控制协议

■ 准确测量往返时间样本的问题

- Karn算法：在原先的RTT估计算法的基础上，对往返时间样本Rts的采样进行了改进

If 收到的**ACK**报文段.**Ack. number**对应的报文段没有重传过 **then**

$$Rtt' = (\alpha \times Rtt) + (1 - \alpha) \times Rts, \quad 0 \leq \alpha < 1,$$

$$Timeout = \beta \times Rtt', \quad \beta > 1$$

Else

$$Timeout' = \gamma \times Timeout, \quad \gamma \geq 2$$

- 非重传采样：对没有二义性的确认进行往返时间采样, 用新样本 Rts 调整 Rtt.
 - RFC 2988 推荐的 α 值为 1/8, 即 0.125, β 推荐值=2
- 重传计时器退避：当发生重传时, 以 $\gamma (\geq 2, \text{推荐值} = 2)$ 因子倍增Timeout, 而不是对往返时间采样



3.4 传输控制协议

■ 准确测量往返时间样本的问题

- RTT基于方差估计算法：1) Karn算法不能适应时延变化很大的情况；2) $\gamma = 2$ 时, Karn算法最多能处理负载为30%的情况

$$Diff = Rts - Rtt$$

$$smoothedRtt = Rtt + \delta \times Diff, \quad 0 \leq \delta \leq 1$$

$$Dev' = Dev + \rho(|Diff| - Dev), \quad 0 \leq \rho \leq 1$$

$$Timeout = smoothedRtt + \eta \times Dev', \quad \eta \geq 1$$

RTT基于方差估计算法,
其中的参数推荐取值为

$$\delta = \frac{1}{2^3}, \rho = \frac{1}{2^2}, \eta = 3$$

参数K称为Kalman gain,
不需要手工设定, K是自学习的

Predict

Kalman Form	Implementation	Equation
$\bar{x} = x + dx$	$\bar{\mu} = \mu + \mu_{f_x}$	$\bar{x} = x + f_x$
$\bar{P} = P + Q$	$\bar{\sigma}^2 = \sigma^2 + \sigma_{f_x}^2$	

Update

Kalman Form	Implementation	Equation
$y = z - \bar{x}$	$y = z - \bar{\mu}$	$x = \ \mathcal{L}\bar{x}\ $
$K = \frac{\bar{P}}{\bar{P} + R}$	$K = \frac{\bar{\sigma}^2}{\bar{\sigma}^2 + \sigma_z^2}$	
$x = \bar{x} + Ky$	$\mu = \bar{\mu} + Ky$	
$P = (1 - K)\bar{P}$	$\sigma^2 = \frac{\bar{\sigma}^2 \sigma_z^2}{\bar{\sigma}^2 + \sigma_z^2}$	

Kalman滤波算法



QUIC协议介绍



■ QUIC协议的目标

- QUIC协议旨在解决传统TCP协议在现代网络环境中的局限性，如连接建立的延迟问题，以及对移动网络和多路复用支持不足的问题。其目标是提供更快的连接建立速度、更低的延迟以及更好的多路复用能力。

■ 传统TCP协议的局限性

- 传统的TCP协议在互联网上广泛使用，但其设计上的一些局限性开始影响现代网络应用的性能。例如，TCP在建立连接时需要三次握手，这增加了延迟。此外，TCP不支持多路复用，导致在高延迟或丢包环境下效率低下。

■ QUIC协议的主要特点

- QUIC协议的主要特点包括快速的连接建立、改进的多路复用能力、以及对前向纠错（FEC）的支持。它还集成了TLS加密，提高了安全性。QUIC通过减少握手次数和优化数据传输机制，显著提升了网络通信的效率。

■ QUIC与TCP/UDP的对比

- 与传统的TCP相比，QUIC在连接建立和数据传输方面更为高效，它减少了握手的往返次数，并支持快速重连。与UDP相比，QUIC提供了类似TCP的可靠传输特性，但增加了对丢包恢复和拥塞控制的改进，使其在保证传输可靠性的同时，也提高了性能



3.4 传输控制协议

■ 3.4.4 TCP滑动窗口与超时重传机制-拓展思考题

1. RTT基于方差估计算法与Karn算法在Timeout计算上有什么区别？
2. 请查阅文献，是否存在基于Bayesian方法估计随机变量RTT？能否使用机器学习方法预测RTT？
3. 请查阅文献，是否存在一种方法在重传情况下也能准确实现往返时间采样？（提示：这是将ACK报文段与第几个重传关联的问题）
4. 请阅读W. Richard Stevens. TCP/IP Illustrated Vol.2 第14.3节，关于Timeout的计算，并分析那里描述的算法与RTT基于方差估计算法有何不同？
5. 请查阅Linux TCP源码
<https://github.com/torvalds/linux/blob/master/net/ipv4/tcp.c>，分析它采用什么算法解决超时重传问题



3.4 传输控制协议

■ 3.4.5 TCP窗口与流量控制、拥塞控制-拓展思考题

1. 拥塞事件的判定是否只能通过推测报文段丢失这一事件？
2. 在非理想情况下，慢开始的拥塞窗口cwnd会怎样变化？
3. TCP Reno版在发生拥塞事件时，直接进入拥塞避免阶段是基于什么原因？
4. 请查阅资料，慢开始在哪些应用场景下性能低下？

